

Universidad de La Habana
Facultad de Matemática y Computación



Unified Multi-Agent Coordination Bridging Large Language Models and Symbolic AI for Autonomous Systems

Autor: Roberto Marti Cedenó

Tutor: [Tutor por confirmar]

Tribunal: [Tribunal por confirmar]

Tesis de Maestría
presentada en opción al título de
Máster en Ciencia de la Computación



Repositorio de la tesis

Commit: [Commit final por registrar]

La Habana, junio de 2026

Resumen

Los modelos grandes de lenguaje pueden interpretar solicitudes y proponer planes, pero la plausibilidad lingüística no establece factibilidad, autoridad ni terminación verificable. Esta tesis propone un límite de autorización y rechazo para coordinar agentes heterogéneos: los componentes lingüísticos producen solicitudes y planes tipados no autoritativos, mientras una capa simbólica resuelve identidades estables, evalúa restricciones, autoridad, dependencias y contratos, y autoriza o rechaza antes de cualquier despacho.

La contribución se presenta como una integración operacional, adaptable a protocolos, de mecanismos establecidos. El prototipo incorpora restricciones tipadas que fallan de forma cerrada, validación completa con JSON Schema, contratos de terminación, ejecución segura respecto a dependencias, trazas persistentes aisladas por sesión, recuperación con arrendamientos y cercas monotónicas, y una política local de admisión A2A. Las verificaciones previas a la congelación apoyan el comportamiento del prototipo en pruebas unitarias, escenarios deterministas, contenedores A2A, una muestra oficial de interoperabilidad y fallos controlados con PostgreSQL. No demuestran verdad semántica, consenso, seguridad bizantina ni superioridad general.

Se implementó además un estudio cegado de 36 casos, tres modelos locales y cinco semillas. La recolección y la puntuación permanecen bloqueadas hasta recibir aprobación independiente de las etiquetas; por ello esta versión no sustituye resultados heredados ni formula conclusiones comparativas prematuras.

Palabras clave: sistemas multiagente; modelos grandes de lenguaje; inteligencia artificial simbólica; autorización; trazabilidad; A2A.

Abstract

Large language models can interpret requests and propose plans, but linguistic plausibility does not establish feasibility, authority, or verifiable completion. This thesis proposes an authorization/refusal boundary for heterogeneous-agent coordination: linguistic components produce typed, non-authoritative requests and plans, while a symbolic layer resolves stable identities, evaluates constraints, authority, dependencies, and contracts, and authorizes or refuses before dispatch.

The contribution is framed as a protocol-adaptable operational integration of established mechanisms. The prototype implements fail-closed typed constraints, complete JSON Schema validation, completion contracts, dependency-safe execution, durable session-isolated traces, lease-and-fencing recovery, and a local A2A admission policy. Pre-freeze checks support prototype behavior in unit tests, deterministic scenarios, containerized A2A checks, one official interoperability sample, and controlled PostgreSQL crash faults. They do not establish semantic truth, consensus, Byzantine security, or general superiority.

A blinded 36-case, three-local-model, five-seed study is also implemented. Collection and scoring remain blocked until independent label approval is supplied; this version therefore neither substitutes legacy results nor states a premature comparative conclusion.

Keywords: multi-agent systems; large language models; symbolic artificial intelligence; authorization; traceability; A2A.

Contents

Resumen	ii
Abstract	iii
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement and Research Gap	1
1.3 Object of Study and Field of Action	2
1.4 Research Objective	2
1.5 Research Questions	3
1.6 Scope and Contributions	3
1.7 Methodological Approach	3
1.8 Thesis Organization	4
2 State of the Art	5
2.1 Framing the Research Problem	5
2.2 Classical Foundations of Autonomous Agents	6
2.3 Multi-Agent Coordination	8
2.4 Agent Communication Languages and Interaction Protocols	9
2.5 Symbolic Foundations	11
2.6 Neuro-Symbolic AI and Hybrid Reasoning	12
2.6.1 Closest Verification-First Planning Work	13
2.7 LLMs as Agent Components	14
2.8 LLM-Based Multi-Agent Systems	15
2.9 Modern Agent Interoperability Protocols	18
2.10 Safety, Verification, and Governance	20
2.11 Evaluation and Benchmarks	21
2.12 Synthesis and Research Gap	22

3	Research Methodology	26
3.1	Study Design	26
3.2	Versioned and Blinded Corpus	26
3.3	Configurations and Fairness	27
3.4	Local Model Protocol	27
3.5	Metrics and Statistical Analysis	27
3.6	Provenance and Reproducibility	28
3.7	Validity Boundaries	28
4	Proposed Theoretical Framework	29
4.1	Purpose and Contribution	29
4.2	Assumptions and Design Requirements	30
4.3	Conceptual Model of Agents and Capabilities	32
4.4	Linguistic Agents and Symbolic Mediation	33
4.5	Bounded Auxiliary Linguistic Capabilities	35
4.6	Coordination, Commitments, and Communication Abstraction	37
4.6.1	Required Communication Properties	37
4.7	Feasibility Model	38
4.8	Properties of the Framework	40
4.9	Worked Example	41
4.10	Limitations and Chapter Synthesis	42
5	System Design: Coordination Agent over SDK Middleware	44
5.1	Instantiation of the Framework	44
5.2	System Components	47
5.3	Agent Registration and Communication Boundary	47
5.4	Minimum Data Structures	49
5.5	Coordinator Workflow	54
5.6	Local and Linguistic Agent Specializations	55
5.7	Architecture Diagram	56
5.8	Execution Sequence	57
5.9	Failure and Infeasibility Handling	57
5.10	Lease-Backed Replicated Coordinator Execution	58
5.11	Chapter Synthesis	60
6	Evaluation and Results	61
6.1	Evidence Status	61
6.2	Fail-Closed Symbolic Boundary	62
6.3	Dependency, Recovery, and Trace Behavior	62

6.4	Corrected End-to-End Harnesses	62
6.5	Comparative Study Gate	63
7	Discussion	64
7.1	Interpretation	64
7.2	Guarantees and Non-Guarantees	64
7.3	Threats to Validity	65
7.4	Defense Readiness	65
8	Conclusions and Future Work	66
A	Reproduction and Artifact Manifest	68
A.1	Command Sequence	68
B	Predicate Refinement and Claim–Evidence Matrix	69
C	Tribunal Questions and Defensible Boundaries	70
	Bibliography	71

List of Figures

4.1	Conceptual boundary of the proposed framework. Linguistic interpretation produces a candidate problem representation; the coordinating authority evaluates it against the session registry and returns either an authorized plan or an explicit infeasibility report. Execution remains distributed among participating agents.	31
4.2	Artifact flow across the linguistic, symbolic, and distributed-execution lanes. Linguistic components produce candidate representations; symbolic validation authorizes a plan or records infeasibility; execution observations return through a structured feedback path.	35
4.3	Task-and-artifact graph for the worked example. The temporary extractor x_1 performs only the bounded Q1/Q2 field selection; registered agents remain responsible for summarization, calculation, and constrained aggregation. . .	41
5.1	CoordinationAgent over SDK middleware for A2A-compatible coordination across remote, local, and linguistic agents.	56
5.2	Execution sequence for CoordinationAgent-mediated problem solving. . . .	57

List of Tables

2.1	Agent architectural traditions relevant to unified coordination.	7
2.2	Closest verification-first work and the remaining thesis distinction.	14
2.3	Representative LLM-based multi-agent approaches.	17
2.4	Emerging interoperability protocols for agent systems.	19
2.5	Evaluation dimensions for unified LLM-symbolic multi-agent systems. . . .	22
2.6	Synthesis matrix deriving requirements for the proposed framework.	23
5.1	Implementation components mapped to theoretical concepts.	48
5.2	Scope of implemented coordination evidence.	59
6.1	Corrected version 0.2 preflight gates.	61

Chapter 1

Introduction

1.1 Context and Motivation

Autonomous systems increasingly coordinate components that differ in ownership, implementation, authority, and reasoning method. Large language models (LLMs) extend these systems with natural-language interpretation, flexible task decomposition, and explanation. Symbolic artificial intelligence (AI), in contrast, provides explicit representations of capabilities, constraints, plans, and state transitions. The two approaches are complementary: language models are effective at proposing meaning under ambiguity, whereas symbolic methods are effective at checking conditions that can be stated precisely.

This complementarity does not automatically yield dependable coordination. An LLM can describe a plausible division of work even when no admitted agent possesses a required capability, two tasks exchange incompatible artifacts, or an action exceeds the authority granted to its executor. Free-form dialogue can also obscure whether an agent merely discussed a task, accepted a commitment, or supplied evidence of completion. These distinctions matter whenever coordinated agents can invoke tools, modify shared state, or affect an external environment.

1.2 Problem Statement and Research Gap

Classical multi-agent systems provide explicit interaction protocols, commitments, organizational roles, and formal coordination mechanisms. Recent LLM-agent systems provide broad linguistic flexibility and tool-use interfaces. Emerging interoperability protocols improve connectivity among independently implemented agents. Nevertheless, these lines of work do not by themselves define when a natural-language coordination proposal is sufficiently justified for execution. In particular, connectivity does not establish semantic compatibility, and fluent agreement does not establish feasibility.

The scientific problem addressed by this thesis is therefore:

How can a heterogeneous multi-agent system use LLM-based interpretation and decomposition while ensuring, through explicit symbolic criteria, that an accepted coordination plan is supported by available capabilities, compatible artifacts, applicable authority, and verifiable completion conditions?

The problem is not merely the construction of another agent platform. It is the absence of a precise operational boundary between a linguistically plausible proposal and a plan that the current system is justified in authorizing. Existing orchestration frameworks, agent protocols, schemas, validators, and tool registries provide important building blocks; the novelty sought here is the enforced combination of those pieces into a coordination boundary where candidate plans, authority checks, artifact contracts, commitments, refusals, and execution traces are all explicit before dispatch.

1.3 Object of Study and Field of Action

The *object of study* is coordination in heterogeneous autonomous multi-agent systems. The *field of action* is the hybrid mediation of natural-language requests into symbolically constrained plans, commitments, execution traces, and infeasibility decisions for independently implemented agents.

1.4 Research Objective

The general objective is to design and specify a unified coordination framework that uses LLMs for semantic interpretation and candidate decomposition while using symbolic representations and deterministic checks to authorize, trace, or reject multi-agent plans.

The specific objectives are:

1. Critically synthesize classical multi-agent coordination, symbolic AI, LLM-agent architectures, and interoperability protocols to identify the unresolved integration gap.
2. Define a protocol-independent theoretical framework for capabilities, tasks, authority, commitments, artifact compatibility, bounded auxiliary capabilities, and feasibility.
3. Instantiate the framework as an implementable Python coordination agent over middleware SDK that preserves protocol-mediated interoperability while supporting remote, local, and Lingo-authored agents without coupling the theoretical contribution to a particular model provider, agent framework, or communication protocol.

4. Evaluate the corrected prototype with a blinded, repeated 36-case study, paired controls and ablations, deterministic scenarios, containerized A2A checks, and measured PostgreSQL crash recovery.
5. Analyze the scope, limitations, validity threats, and practical implications of the proposed framework.

1.5 Research Questions

The thesis is guided by the following questions:

- RQ1** Which coordination information must be represented explicitly to distinguish an executable multi-agent plan from a linguistically plausible one?
- RQ2** How should responsibility be divided between LLM-based reasoning and symbolic validation in a heterogeneous coordination workflow?
- RQ3** How can missing narrow linguistic capabilities be introduced without granting unbounded new competence or authority?
- RQ4** What do the measured outcome and process results show about justified acceptance, explicit refusal, traceability, dependency discipline, and recovery relative to paired controls and ablations?

1.6 Scope and Contributions

The thesis focuses on plan-level coordination under a trusted central authorization role and distributed execution. It does not claim to verify the internal reasoning of participating agents, solve Byzantine coordination, or guarantee runtime success after authorization. Its contribution is integrative: a formal separation between linguistic proposal and symbolic authorization; typed, fail-closed feasibility and completion contracts; bounded task-local auxiliary capabilities; durable trace and measured crash recovery; a local A2A admission boundary; and a reproducible method for comparing the integration with paired controls. Established component mechanisms, the new operational integration, prototype evidence, and unproven properties are kept distinct.

1.7 Methodological Approach

The research follows a design-science approach. It combines critical literature synthesis, formal and architectural modeling, implementation-oriented specification, and controlled

evaluation. The evaluation separates planning feasibility from runtime success, uses blinded labels and repeated local-model runs, tests mechanism ablations, and measures replicated-coordinator ownership through a shared durable store. Chapter 3 explains the executed and gated stages.

1.8 Thesis Organization

Chapter 2 synthesizes the theoretical foundations and research gap. Chapter 3 defines the research and evaluation method. Chapter 4 presents the protocol-independent framework. Chapter 5 maps it to a coordination-agent architecture over SDK middleware and then selects a concrete interoperability adapter for implementation. Chapter 6 specifies the evaluation execution and reporting structure. Chapter 7 examines the implications, limitations, and validity threats. The final chapter relates the contributions to the objectives and identifies future work.

Chapter 2

State of the Art

2.1 Framing the Research Problem

The contemporary discussion of artificial intelligence agents is marked by a productive but sometimes imprecise convergence between two research traditions. The first tradition is the classical computer-science study of autonomous agents and multi-agent systems, in which an agent is a situated computational entity that perceives an environment, reasons over goals or policies, and acts through effectors [106, 85, 52]. The second tradition is the recent development of large language model (LLM) applications that use natural language, tool invocation, memory, and planning prompts to produce behavior that is described as agentic [14, 102, 109, 101]. The overlap between these traditions is substantial, but not identical. Classical agents emphasize autonomy, reactivity, proactiveness, social ability, commitments, and environment interaction; LLM agents often emphasize linguistic reasoning, flexible task decomposition, and interaction with external tools.

This distinction is central to a thesis on unified multi-agent coordination. A chatbot that produces a response to a prompt is not, by itself, an autonomous agent in the classical sense. Conversely, a symbolic controller that follows a plan can be an agent even if it has no natural-language interface. The research opportunity appears when LLMs are treated as components inside broader agent architectures rather than as replacements for agency itself. In that view, an LLM can serve as a linguistic reasoning module, a translator between human instructions and formal actions, an explanation generator, or a coordinator of high-level intent. The agent, however, remains the larger computational system that maintains state, chooses actions, interacts with other agents, and is accountable for execution.

The literature therefore motivates a bridging problem. Classical multi-agent systems (MAS) provide mature formalisms for interaction protocols, negotiation, task allocation, organizational roles, commitments, and verification [105, 103, 25, 93]. LLM-based systems provide flexible natural-language mediation, few-shot generalization, and tool-use interfaces [78, 87, 68]. Symbolic AI supplies explicit representations for planning, constraint

satisfaction, rule-based control, and reasoning over dynamic worlds [37, 84, 45]. A unified coordination architecture must integrate these strengths without collapsing their differences. In particular, it must support agents that communicate linguistically, reason symbolically, invoke tools safely, and participate in multi-agent coordination mechanisms whose semantics are explicit enough to be analyzed.

The phrase state of the art is used here in a precise sense. It does not refer only to the most recent LLM frameworks or protocols, because novelty alone is not sufficient for autonomous coordination. Instead, the relevant state of the art consists of the strongest available concepts, formalisms, systems, and evaluation practices that inform the design of a coordinated autonomous system. This includes older but still foundational work on agency and communication, mature symbolic methods for action and verification, and newer LLM-based systems that demonstrate flexible linguistic control. The purpose of the chapter is therefore synthetic: it identifies what each research line contributes, where each line remains insufficient, and why a unified LLM-symbolic MAS architecture is a coherent research target.

2.2 Classical Foundations of Autonomous Agents

The classical agent literature provides the conceptual baseline for this thesis. Wooldridge and Jennings characterize agents through autonomy, social ability, reactivity, and proactiveness, thereby distinguishing agents from passive objects or services [106]. Russell and Norvig formulate the rational-agent view, in which an agent maps percept histories to actions and is evaluated by a performance measure in an environment [85]. These definitions remain important because they locate agency in a control loop: perception, internal state update, decision, and action. The same loop is required for autonomous systems whether their reasoning module is symbolic, neural, or hybrid.

Several architectural families have shaped the design of autonomous agents. Reactive agents connect percepts to actions through condition-action rules or layered behaviors. Brooks’s subsumption architecture demonstrated that robust situated behavior could arise from distributed reactive layers without a centralized symbolic world model [13]. Deliberative agents, by contrast, maintain explicit models and use search or planning to choose actions. Hybrid architectures combine these approaches, usually by placing a fast reactive layer under a slower deliberative layer. Cognitive architectures such as Belief-Desire-Intention (BDI) systems add explicit mental attitudes, representing what the agent believes, wants, and has committed to doing [12, 82, 83, 35].

BDI architectures are particularly relevant to the present thesis because they provide a disciplined vocabulary for autonomous behavior. Beliefs encode information about the world, desires encode possible objectives, and intentions encode commitments to plans. This separation is useful for LLM-symbolic systems because LLM output should not automatically

become a commitment to act. A linguistic proposal may be treated as a candidate desire, an explanation, or a plan sketch; the symbolic or procedural layer can then validate whether it should become an intention. This division helps address a common weakness of LLM agents: fluent text can appear decisive even when the underlying action has not been checked against state, preconditions, or policy.

Agent-oriented programming also contributes important design principles. Shoham proposed programming in terms of mentalistic concepts such as beliefs, obligations, and capabilities [91]. Later platforms such as JADE, Jason, GOAL, and JaCaMo operationalized agent communication, BDI reasoning, and multi-agent organization in software frameworks [7, 11, 43, 9]. These systems show that agent design benefits from explicit abstractions for messages, roles, plans, and norms. Modern LLM frameworks often rediscover these abstractions through role prompts, tool registries, memory modules, and conversation controllers, but the classical literature provides more precise semantics for them.

The boundary between agent and non-agent software is therefore not merely terminological. A database, an API, a workflow, and a model endpoint can all participate in an agentic system, but they do not necessarily exhibit agency. An agent must have a decision procedure and some degree of autonomy with respect to its environment [32]. This matters for system design because different components require different forms of assurance. A tool can be validated by its interface contract; a model can be evaluated by prediction or generation quality; an autonomous agent must also be evaluated by its policy for choosing when and how to act. In LLM-based systems, this distinction is often obscured when every tool wrapper or prompt template is called an agent. A rigorous thesis architecture should reserve the term agent for components that participate in a sense-reason-act loop with explicit state and responsibility.

Table 2.1: Agent architectural traditions relevant to unified coordination.

Tradition		Relevance to this thesis
Reactive agents		Provide fast environment response and motivate safety shields that can override slower deliberation.
Deliberative agents		Provide explicit planning, goal reasoning, and action selection over symbolic state.
BDI agents		Separate beliefs, desires, and intentions, which is useful for interpreting LLM proposals before execution.
Hybrid agents		Combine reactive control with deliberative reasoning, matching the need to combine LLM reasoning with symbolic execution.
Agent-oriented programming	pro-	Provides software abstractions for roles, commitments, messages, plans, and organizational structure.

2.3 Multi-Agent Coordination

Multi-agent systems extend the agent concept from individual autonomy to distributed computation among multiple autonomous entities. In a MAS, agents may cooperate, compete, negotiate, delegate, or coordinate around shared resources [25, 103, 105]. The key difficulty is that no single agent necessarily possesses complete knowledge, global control, or unlimited authority. Coordination is therefore required to manage dependencies among activities, allocate tasks, resolve conflicts, synchronize plans, and maintain coherent collective behavior. Malone and Crowston define coordination broadly as the management of dependencies between activities [65]; the MAS literature operationalizes this idea through protocols, organizational structures, distributed algorithms, and shared environments [51].

Task allocation is one of the oldest and most concrete coordination problems. The Contract Net Protocol introduced a manager-contractor pattern in which tasks are announced, agents bid, and contracts are awarded according to suitability or cost [94]. Market-based coordination generalizes this approach by using auctions, marginal costs, and economic mechanisms to allocate work or resources [86, 104]. Coalition formation addresses cases in which a task requires the combined capabilities of multiple agents [89]. These mechanisms are directly relevant to LLM-based multi-agent systems, where role-specialized agents may be dynamically selected for subtasks such as planning, coding, verification, retrieval, or simulation.

Shared-state coordination offers a different model. Blackboard systems allow specialized knowledge sources to read and write partial solutions in a common workspace [76, 18]. Tuple-space coordination, exemplified by Linda, decouples agents through generative communication over shared tuples [33]. Stigmergic coordination uses changes in the environment as indirect signals, a concept developed in studies of social insects and later applied to swarm systems [96]. These mechanisms anticipate modern agent workspaces, shared memories, vector stores, and artifact repositories. They also expose a risk: if the shared state is ambiguous or corrupted, coordination can fail even when each individual agent appears locally competent.

Distributed constraint reasoning provides a more formal approach to joint decision-making. Distributed Constraint Satisfaction Problems and Distributed Constraint Optimization Problems model decisions as variables controlled by different agents under local and global constraints [110, 73, 28]. Such formalisms are valuable for autonomous systems because they make resource limits, task dependencies, and optimization criteria explicit. LLM systems rarely use DCOP-style models directly, but a unified architecture can use LLMs to elicit goals or preferences while a symbolic optimizer enforces constraints. This division is especially important when natural-language instructions are incomplete, inconsistent, or underspecified.

Social and normative approaches broaden coordination beyond efficiency. Commitments, norms, institutions, trust, and argumentation provide mechanisms for regulating interactions among autonomous agents [93, 22, 81]. In classical MAS, a commitment can represent an obligation from one agent to another; protocols can then be evaluated in terms of how commitments are created, discharged, or violated. This perspective is relevant to LLM agents because dialogue alone does not guarantee accountability. If one agent asks another to perform an action, a unified system should represent whether that request created an obligation, whether the target accepted it, and what evidence establishes completion.

Coordination failure is as important as coordination success. Distributed systems may fail through deadlock, livelock, inconsistent local beliefs, resource contention, duplicated effort, or incompatible goals. LLM-based MAS add further failure modes: agents may agree linguistically while using different internal interpretations; a critic agent may approve an artifact without checking it; multiple agents may reinforce the same hallucination; or a delegated task may be considered complete because the conversation ended rather than because a verifiable postcondition was reached. Classical MAS mechanisms provide vocabulary for diagnosing these failures. For example, a protocol trace can reveal that an expected acceptance message was never sent, and a commitment record can reveal that a promised action lacks evidence of completion. This is a strong argument for storing coordination state in explicit symbolic structures rather than only in chat history.

The distinction between centralized and decentralized coordination also requires careful treatment. Centralized orchestration simplifies global control and is common in practical LLM systems, where a controller routes messages among role-specific agents. Decentralized coordination can improve robustness and scalability, but it requires stronger assumptions about local decision-making and communication semantics. Hybrid coordination is often more realistic: a central orchestrator may allocate tasks, while agents negotiate details, update shared artifacts, and call local tools. For autonomous systems, the correct choice is domain-dependent. Safety-critical execution may require centralized authorization, while exploratory planning may benefit from decentralized proposal generation and critique.

2.4 Agent Communication Languages and Interaction Protocols

Coordination depends on communication, but not all communication has the same semantics. Classical Agent Communication Languages (ACLs) were developed to provide structured, interoperable message exchange among agents. Their theoretical basis lies in speech-act theory, where utterances are understood as actions that can request, inform, promise, or commit [5, 88, 16]. KQML introduced performatives such as ask, tell, and reply, together with

message fields for content, language, and ontology [27, 56]. FIPA-ACL later standardized communicative acts and interaction protocols with more explicit semantics grounded in mental attitudes [30].

The importance of ACLs for this thesis is not that modern systems should necessarily reuse KQML or FIPA unchanged. Rather, the key lesson is that coordination messages require interpretable force. A statement such as "the route is clear" differs from "please clear the route" or "I commit to clearing the route." In natural language, such distinctions are often implicit. LLM agents can generate and interpret nuanced language, but they can also blur assertion, request, plan, and commitment. A unified coordination architecture should therefore map natural-language utterances to structured communicative acts when those utterances affect execution.

Interaction protocols provide the temporal structure of communication. Contract net, request-response, subscription, negotiation, and auction protocols specify allowed message sequences and expected state transitions [30, 53]. In classical MAS, this structure supports interoperability and verification. In LLM-based systems, conversation often proceeds through ad hoc chat transcripts. Such transcripts are flexible, but flexibility can make it difficult to determine whether a task has been accepted, delegated, blocked, or completed. A hybrid architecture can preserve natural-language flexibility for explanation and negotiation while retaining structured protocol state for execution.

Content languages and ontologies also remain central. Gruber defines an ontology as an explicit specification of a conceptualization [40]; description logics provide formal foundations for class, relation, and constraint reasoning [6]. Without shared semantics, agents may exchange syntactically valid messages that have different meanings to different participants. LLMs can help mediate heterogeneous vocabularies, but they cannot by themselves guarantee semantic alignment. For autonomous systems, especially those controlling tools or physical processes, semantic grounding must be explicit enough to support validation.

The transition from ACLs to natural-language coordination should therefore be understood as an expansion of expressive capacity rather than a replacement for structure. Natural language is valuable because it can express uncertainty, context, explanation, and negotiation without requiring all possibilities to be pre-enumerated. However, execution requires discrete commitments. An LLM may write, "I can handle the scheduling task," but a coordination layer must determine whether this is an assertion of capability, an offer, an acceptance, or a mere conversational signal. Classical ACLs solve this by using performatives; modern LLM systems can approximate the same effect by extracting structured message types, confidence, referenced artifacts, intended recipients, and expected follow-up actions. The extraction need not eliminate natural language; it should accompany it with machine-checkable coordination metadata.

Another issue is observability. In a free-form conversation, the state of the protocol is

often implicit in the transcript. A human reader may infer that a task has moved from proposal to execution, but a verifier cannot safely depend on such inference. Structured communicative acts make protocol state observable. For example, a request can be represented with a sender, receiver, content, deadline, acceptance status, and completion criterion. This explicit state is compatible with LLM-generated messages, but it requires an architectural layer that treats dialogue as input to a coordination model rather than as the coordination model itself.

2.5 Symbolic Foundations

Symbolic AI provides the formal machinery needed to represent knowledge, define actions, and reason about consequences. Early work on logic and common-sense reasoning established the idea that intelligent behavior can be supported by symbolic descriptions of the world and action [66, 75]. Reiter’s work on knowledge in action developed logical foundations for specifying dynamic systems [84]. Knowledge graphs and ontologies extend this tradition by supporting semantic integration across heterogeneous data sources [45]. In a unified agent architecture, these symbolic representations can serve as the stable substrate against which LLM-generated interpretations are checked.

Automated planning is especially relevant because it provides a disciplined account of goal-directed action. STRIPS introduced actions with preconditions and effects [26]; PDDL later provided a standard planning-domain language that supported interoperability among planners [67, 31]. Planning systems search for action sequences that transform an initial state into a goal state under domain constraints [37]. This formalism directly addresses a weakness of LLM planning: language models can produce plausible plans without verifying action applicability, resource limits, or causal dependencies. A planner can reject or repair such plans when the formal model is available.

Hierarchical Task Network (HTN) planning adds a decomposition perspective. Rather than planning only through primitive actions, HTN methods refine abstract tasks into subtasks according to domain-specific methods [24, 74, 36]. This is closely aligned with how LLM agents often decompose natural-language tasks into steps. The difference is that HTN decomposition is constrained by explicit methods, whereas LLM decomposition is often implicit in generated text. Combining the two can allow an LLM to propose a decomposition while a symbolic layer verifies that each refinement is permitted by the domain model.

Rule-based and non-monotonic reasoning also contribute to agent control. Logic programming and Answer Set Programming support declarative encodings of constraints, defaults, and alternative stable models [34, 62]. These approaches are useful when coordination requires reasoning about exceptions, incomplete knowledge, or mutually exclusive choices. Finite-state machines, behavior trees, and Goal-Oriented Action Planning provide additional symbolic control structures that are widely used in robotics and games [17, 77]. Their

advantage is not linguistic flexibility but predictable execution structure. In safety-critical autonomous systems, predictability is not optional.

Symbolic methods also clarify the difference between a plan and an explanation. LLMs can explain a possible course of action in coherent prose, but a plan in the symbolic sense is an object with preconditions, effects, ordering constraints, resources, and success criteria. Treating explanations as plans can lead to execution errors because the text may omit necessary conditions or represent them ambiguously. Conversely, symbolic plans can be difficult for humans to understand without explanation. A hybrid system can use a symbolic planner to produce or verify the executable structure while using an LLM to explain the plan, ask for missing information, or translate user intent into formal goals. The two artifacts should remain linked but distinct.

There are also limits to symbolic methods. A planner can only reason over what has been modeled. A rule engine can only enforce rules that are encoded. A knowledge graph may be incomplete or stale. The role of the LLM is therefore not only convenience; it can help interpret underspecified tasks, retrieve relevant context, and generate candidate formalizations. The strongest hybrid design is not a symbolic system with a superficial language interface, nor an LLM system with occasional tool calls. It is an architecture that assigns responsibilities according to epistemic strength: language models handle interpretation and communication; symbolic components handle explicit constraints, commitments, and execution semantics.

2.6 Neuro-Symbolic AI and Hybrid Reasoning

Neuro-symbolic AI studies the integration of neural learning with symbolic reasoning. Surveys emphasize that neural systems are effective for perception, pattern recognition, and language, while symbolic systems support abstraction, compositionality, explanation, and formal reasoning [8, 20, 44]. LLMs intensify this integration problem. They are neural models with broad linguistic competence, but their outputs are not inherently grounded in formal semantics. A hybrid architecture can exploit their strengths while delegating constraint satisfaction, planning, and verification to symbolic components.

One important pattern is LLM-to-symbolic translation. A language model may translate a user request into a PDDL goal, a program, a query, a constraint set, or an action schema. LLM+P, for example, explores the use of LLMs with classical planning by translating natural-language planning tasks into planner-compatible representations [63]. Studies of LLM planning capabilities also show that LLMs are not reliable substitutes for planners when exact action semantics are required [98]. The implication is not that LLMs are unhelpful for planning; rather, they should be placed where linguistic flexibility is valuable and checked where formal correctness matters.

A second pattern is symbolic-to-LLM augmentation. Retrieval-augmented generation

supplies external documents as context [58]; tool-augmented systems expose calculators, search engines, databases, or APIs [87, 68]; MRKL systems propose modular architectures in which language models route tasks to specialized symbolic or computational modules [55]. In agent systems, this pattern generalizes to planners, solvers, knowledge graphs, monitors, and execution kernels. The LLM becomes an interface and orchestrator, not the sole reasoner.

The most relevant pattern for this thesis is a hybrid execution loop. The agent perceives or receives a task, uses an LLM to interpret context and propose candidate actions, translates selected candidates into formal representations, uses symbolic modules to validate or refine them, executes through a controlled interface, and records feedback for future reasoning. This loop aligns with ReAct-style interleaving of reasoning and action [109], but adds a stronger separation between linguistic proposal and executable commitment. Such separation is essential in multi-agent settings because one agent’s text may trigger another agent’s action. The bridge between language and symbolic execution must therefore include validation, provenance, and state updates.

2.6.1 Closest Verification-First Planning Work

Formal-LLM supervises LLM plan generation with a developer-supplied automaton, directly addressing invalid plan sequences [61]. Metagent-P combines LLM world knowledge, symbolic cognitive architecture, and metacognitive verification for open-world planning [114]. Verified Multi-Agent Orchestration uses a plan–execute–verify–replan loop and an LLM-based completeness verifier over a question DAG [111]. The formal-methods position of Zhang et al. argues more broadly that trustworthy agents require integration between LLMs and formal methods [112]. These works make a generic “neuro-symbolic” novelty claim untenable; the remaining gap is narrower and operational.

The bridge is not a single translator. It is better understood as a set of transformations and checks. Natural-language goals may be transformed into symbolic goals; symbolic plans may be transformed into natural-language explanations; tool schemas may be transformed into LLM-callable functions; monitor violations may be transformed into feedback for replanning. Each transformation can introduce error. Consequently, a robust bridge should record provenance, expose confidence or uncertainty when available, and support rejection or repair. This position differs from treating the LLM as a universal planner. It treats the LLM as one participant in a heterogeneous reasoning pipeline.

Hybrid reasoning also raises a granularity question. The LLM may operate at the level of high-level tasks, while a symbolic planner operates at the level of primitive actions. If the boundary is too high, the symbolic layer cannot catch relevant failures. If the boundary is too low, the LLM may be forced to produce brittle formal details it is not well suited to generate. A practical architecture should allow multiple levels of representation: strategic

Table 2.2: Closest verification-first work and the remaining thesis distinction.

Work	Primary verification mechanism	Distinction of this thesis
Formal-LLM	Automaton-constrained plan generation	Authorizes composition against a live admitted registry, local authority, artifact contracts, and durable attempt evidence.
Metagent-P	Cognitive-symbolic planning plus metacognitive verification	Focuses on heterogeneous executor admission, refusal, protocol adaptation, and dependency-safe execution rather than one agent’s open-world planning loop.
Verified Multi-Agent Orchestration	LLM verification of completeness followed by replanning	Uses deterministic plan-level gates for modeled predicates and distinguishes contract validity from LLM quality judgment.
Trustworthy-agent formal-methods position	Research agenda joining LLMs and formal methods	Supplies one concrete implementation refinement, crash-recovery boundary, and reproducibility protocol; it does not claim comprehensive formal verification.

goals, decomposed tasks, protocol messages, symbolic constraints, and executable tool calls. Coordination then becomes the process of maintaining consistency across these levels.

2.7 LLMs as Agent Components

The rise of transformer-based language models created new possibilities for agent design. The transformer architecture enabled scalable sequence modeling through attention mechanisms [99]. Large pre-trained models demonstrated few-shot and instruction-following behavior [14, 78]. Chain-of-thought prompting showed that intermediate reasoning traces can improve performance on some reasoning tasks [102]. These developments made it practical to use LLMs as flexible reasoning and communication modules inside larger systems.

ReAct is an influential pattern because it interleaves reasoning traces with actions over external tools or environments [109]. Toolformer and related work further demonstrate that language models can be adapted to call external tools [87]. Augmented language model surveys describe this broader movement toward coupling LLMs with retrieval, tools, memory, and external computation [68]. For autonomous agents, tool use changes the risk profile. A generated sentence may be harmless; a generated tool call can modify files, execute code, send messages, or change a physical state. Therefore, the agent architecture must treat tool invocation as an execution event governed by permissions, validation, and monitoring.

Memory is another key capability. Retrieval-augmented generation provides access

to external knowledge [58]; Reflexion introduces verbal feedback and episodic memory to improve future behavior [90]; Voyager stores reusable skills learned in a Minecraft environment [100]. These approaches show that agent performance depends not only on the base model but also on persistent context and learned artifacts. In a unified architecture, memory should be divided into forms with different semantics: conversational memory, factual knowledge, symbolic state, plan history, tool traces, and commitments. Collapsing all memory into unstructured text increases flexibility but reduces verifiability.

Embodied and interactive agents further illustrate both the promise and limits of LLM-based agency. Work on language models as zero-shot planners and embodied reasoning explores how LLMs can produce action sequences for simulated or physical tasks [48, 49]. Voyager demonstrates open-ended exploration with a skill library [100]. Generative Agents simulate social behavior through memory, reflection, and planning [79]. These systems show that LLMs can support long-horizon behavior, but they also depend on scaffolding: memory retrieval, environment feedback, tool execution, and action validation. The agent is the full scaffolded system, not only the model.

The current LLM-agent literature also reveals a tension between generality and reliability. Generality comes from representing many tasks in language and relying on the model’s broad prior knowledge. Reliability usually comes from narrowing the action space, adding tests, constraining tools, and validating intermediate artifacts. Autonomous systems need both. A useful agent may have to interpret open-ended human instructions, but it should not execute open-ended actions without controls. This tension explains why tool schemas, sandboxes, structured outputs, retrieval systems, and evaluators have become common in agent frameworks. They are attempts to recover reliability around a flexible linguistic core.

For multi-agent coordination, another limitation is social reasoning. LLMs can simulate dialogue and infer intentions, but simulated agreement does not necessarily correspond to coordinated execution. A planning agent may believe a worker agent has accepted a task because the worker responded positively, while the worker may lack the capability or authority to execute it. Classical MAS concepts such as capabilities, commitments, and organizational roles can reduce this ambiguity. The LLM can make role-based interaction more natural, but the system should still maintain explicit records of who can do what, who has promised what, and what evidence resolves the promise.

2.8 LLM-Based Multi-Agent Systems

LLM-based multi-agent systems adapt MAS ideas to teams of language-model-driven components. Surveys of LLM agents and LLM-based MAS identify common patterns: role specialization, debate, delegation, reflection, tool use, and conversation-based collaboration [101, 42, 97]. These systems often assign roles such as planner, critic, coder, reviewer,

researcher, or executor. The roles are usually encoded in prompts rather than in formal organizational models, but the functional resemblance to classical MAS is clear.

AutoGen introduced a flexible framework for composing conversable agents that may use LLMs, tools, or humans in the loop [107]. CAMEL uses role-playing prompts to study communicative cooperation among agents [60]. MetaGPT encodes software-engineering workflows as standard operating procedures distributed across specialized agents [46]. Chat-Dev similarly frames software development as communication among role-specialized agents [80]. Current engineering frameworks broaden this pattern: LangGraph provides low-level orchestration for durable, stateful agents [57]; Semantic Kernel provides middleware for integrating AI agents, models, plugins, and application code [70]; CrewAI organizes autonomous agents into crews and flows for complex workflows [19]. These systems demonstrate that multi-agent decomposition can improve task structure, distribute responsibilities, support persistence, and enable iterative critique. However, they do not by themselves settle whether a natural-language plan is authorized by an admitted registry, whether artifact contracts are semantically sufficient, or whether a refusal should be returned before dispatch.

Lightweight conversational-modeling frameworks address a narrower but important part of this landscape. Lingo is a type-safe Python framework for building LLM-powered applications around explicit conversational state, structured generation, structured flows, finite-state control, reflexive guardrails, and developer-controlled tool loops [41]. Its relevance is not that it replaces a MAS platform, an interoperability protocol, or a symbolic planner. Rather, it shows a state-of-the-art pattern for disciplining the linguistic layer itself: free-form language is wrapped in typed context, constrained model decisions, schema-validated outputs, and state-conditioned dialogue. This makes Lingo a useful example of how linguistic agents can be engineered inside a larger autonomous system while still requiring external authorization and feasibility validation before generated proposals become executable commitments.

This framework landscape clarifies the difference between a linguistic-agent construction toolkit and a coordination architecture. The former can narrow model behavior into typed acts, support state-conditioned dialogue, and expose guardrail hooks; the latter must still decide which agents may act, whether a plan is feasible, how artifacts satisfy contracts, and what communication trace establishes completion. For this thesis, the SOTA implication is that language-mediated flexibility should be retained, but it should be placed behind explicit contracts, protocol state, and symbolic authorization.

The main lesson from this literature is that LLM-MAS research has reintroduced coordination as a practical necessity. When multiple LLM agents interact, the system needs role assignment, message routing, shared memory, conflict resolution, and evaluation of partial results. These are classical MAS problems. What is new is the linguistic richness of the agents and their ability to transform vague human instructions into candidate artifacts. The challenge is to connect that richness with formal models of interaction, state, and execution.

Table 2.3: Representative LLM-based multi-agent approaches.

Approach	Coordination pattern	Limitation motivating hybrid design
AutoGen	Programmable conversations among agents, tools, and humans.	Conversation flexibility does not by itself provide formal protocol semantics.
CAMEL	Role-playing agents cooperate through prompted dialogue.	Role behavior is prompt-dependent and difficult to verify.
MetaGPT	Specialized agents exchange structured artifacts within a workflow.	Standard operating procedures require domain design and may remain task-specific.
ChatDev	Software development is decomposed into communicative roles.	Success depends on artifact consistency and evaluation of generated code.
LangGraph, Semantic Kernel, CrewAI	Practical orchestration, middleware, crews, flows, tools, and durable state for LLM applications.	Provide useful engineering substrates, but symbolic feasibility, authorization ownership, and infeasibility semantics remain application-level responsibilities.
Lingo	Typed conversational modeling with context, structured generation, flows, state machines, and reflexive guardrails.	Provides a disciplined linguistic layer, but not distributed MAS protocol semantics or symbolic feasibility checks.

Without such grounding, multi-agent dialogue may produce impressive transcripts but weak guarantees.

A second lesson is that role prompting is not the same as role semantics. In many LLM-MAS systems, an agent is assigned a role through natural-language instructions such as "act as a software architect" or "act as a reviewer." This can be effective for shaping model behavior, but it does not define a role in the organizational sense. A formal role has permissions, responsibilities, expected messages, authority boundaries, and accountability conditions. The gap between prompted roles and formal roles is significant for autonomous systems. A reviewer prompt may encourage criticism, but a reviewer role in a verified workflow should specify what evidence must be checked before approval.

The literature also suggests that multi-agent decomposition can sometimes improve output quality while also increasing system complexity. More agents mean more messages, more state transitions, more opportunities for inconsistency, and higher computational cost. The benefit of additional agents should therefore be justified by functional separation: one agent may specialize in natural-language interpretation, another in symbolic planning, another in verification, and another in execution. This division aligns with heterogeneous MAS more than with homogeneous debate among multiple copies of the same model. The thesis problem is thus not merely how to make many LLMs talk, but how to coordinate different kinds of reasoning entities.

2.9 Modern Agent Interoperability Protocols

The recent growth of agent frameworks has produced a new interoperability problem. Agents built in different frameworks may need to access tools, exchange context, advertise capabilities, delegate tasks, or collaborate across organizational boundaries. Several emerging protocols address parts of this problem. As of July 2026, the Model Context Protocol (MCP) defines a standardized client-server boundary for tools, resources, prompts, roots, sampling, elicitation, transports, and authorization; the thesis treats the 2025-11-25 specification as the stable cited baseline and the current draft page as protocol-evolution evidence rather than as a settled production guarantee [4, 71, 72]. Agent2Agent (A2A) focuses on communication and interoperability between opaque agentic applications, including Agent Card discovery, JSON-RPC over HTTP(S), streaming, push notification, and structured data exchange; it is an open-source project under Linux Foundation governance, contributed by Google [38, 1, 95]. The Agent Communication Protocol (ACP) was introduced as an HTTP-native agent communication standard, but its documentation and LF AI and Data describe ACP as part of A2A, with independent active development winding down [50, 2, 59]. The Agent Network Protocol (ANP) emphasizes decentralized identity, discovery, messaging, and layered agent-network interoperability [3, 15].

Table 2.4: Emerging interoperability protocols for agent systems.

Protocol	Primary focus	Relevance to unified coordination
MCP	Tool and context integration between AI applications and external systems.	Provides a standardized tool/context boundary, but does not by itself define full agent-to-agent social semantics.
A2A	Agent-to-agent communication, capability discovery, and collaboration.	Addresses cross-framework agent interaction and can support higher-level coordination protocols.
ACP	Lightweight HTTP-native agent communication, now aligned with A2A efforts.	Illustrates convergence toward shared messaging standards for heterogeneous agents.
ANP	Decentralized identity, discovery, and networked agent communication.	Highlights identity and discovery as first-class concerns in open agent ecosystems.

These protocols should be interpreted carefully. They are important engineering developments, but they are not complete replacements for classical ACL semantics [23]. MCP can expose tools to an LLM application, but it does not determine whether a tool call is a request, a delegation, a commitment, or a verified action. A2A can support agent discovery and communication, but application-level coordination still requires domain protocols and state models. ACP’s convergence into A2A also shows that the protocol landscape is consolidating, not that the semantic coordination problem has disappeared. The relevant research question is therefore how modern protocol infrastructure can be mapped onto classical MAS concepts such as performatives, commitments, roles, and interaction protocols. A thesis architecture that bridges LLMs and symbolic AI should use modern protocols for interoperability while preserving explicit semantics for coordination and execution.

The protocol landscape also illustrates the difference between connectivity and coordination. Connectivity means that components can exchange messages or expose tools. Coordination means that those exchanges contribute to a coherent joint process with interpretable state and controlled effects. MCP improves the tool boundary by making external context and tools available through a common interface. A2A and related efforts address cross-agent messaging and capability discovery. These are necessary infrastructure layers, but a MAS still requires policies for when to delegate, how to evaluate capability claims, how to recover from failure, and how to resolve conflicting commitments. In this sense, modern protocols lower the cost of integration but do not remove the need for agent theory.

Protocol evolution is also a governance issue. If agents are expected to operate across organizations, they must advertise identity, capabilities, permissions, and trust assumptions. Decentralized discovery may support open ecosystems, but it also creates risks around impersonation, malicious agents, and incompatible policy regimes. A unified architecture should therefore separate protocol transport from trust decisions. It should be possible to discover an agent without automatically trusting it, to receive a request without accepting it, and to expose a tool without granting unrestricted authority. These distinctions are familiar in distributed systems and security engineering, but they must be made explicit in LLM-agent architectures.

2.10 Safety, Verification, and Governance

Safety and verification are central because autonomous agents can produce effects beyond text generation. Classical MAS and autonomous-systems research has long examined model checking, runtime verification, ethical decision constraints, and verification of agent programs [29, 10, 21]. These techniques become more important when LLMs participate in action selection. A language model may produce a plausible action that violates a precondition, exceeds authority, leaks information, or conflicts with another agent’s commitment. Symbolic verification cannot eliminate all risks, but it can check well-specified properties of plans, tool calls, and execution traces.

Runtime monitoring is a practical complement to offline verification. A monitor can enforce invariants over tool calls, memory writes, inter-agent messages, and environment actions. For example, a monitor may prevent an agent from executing a destructive operation without authorization, writing to protected memory, or sending a message that violates a protocol state. Such mechanisms fit naturally with a hybrid architecture: LLMs propose and explain, symbolic controllers validate and execute, and monitors observe the boundary between proposal and action.

LLM-specific safety risks include hallucination, prompt injection, tool misuse, data leakage, and manipulation of shared memory. Indirect prompt injection shows that external content can influence tool-using LLM applications in ways not intended by the user or developer [39]. Security analyses of MCP ecosystems emphasize that tool integration expands the attack surface for AI applications [47]. In multi-agent settings, these risks can propagate: one compromised agent may poison shared memory, issue misleading messages, or delegate unsafe actions. Therefore, governance must include identity, permissions, audit logs, provenance, and explicit boundaries between agents.

The safety literature supports a defense-in-depth position. Natural-language policies are useful for communication, but enforceable policies require structured representations. Human oversight is important, but scalable systems also need automated checks. LLM self-critique

can identify some errors, but independent symbolic validation is more reliable for formal constraints. For this reason, the state of the art points toward architectures where language models are powerful but bounded participants in a larger verified control system.

Verification must also be scoped realistically. Full verification of an open-ended LLM is not currently practical. However, many safety-relevant properties concern the symbolic envelope around the model rather than the model weights themselves. A system can verify that a tool call conforms to a schema, that a plan action satisfies a precondition, that a workflow state permits a transition, that a user has authorized a capability, or that a message follows a protocol. These checks do not prove that the LLM is truthful or safe in all contexts, but they reduce the space in which model errors can produce harmful effects. The thesis can therefore pursue practical assurance by verifying the controllable parts of the agent loop.

Governance also requires traceability. A coordinated system should record why an action was selected, which agent proposed it, which module validated it, which policy authorized it, and what result followed. Such traces support debugging, evaluation, and accountability. They also support scientific analysis: without traces, it is difficult to determine whether success came from genuine coordination, accidental model knowledge, or hidden human intervention. For a master’s thesis solution, traceability is valuable both as an engineering feature and as an empirical research instrument.

2.11 Evaluation and Benchmarks

Evaluation of unified LLM-symbolic multi-agent systems requires multiple dimensions. Classical MAS evaluation considers coordination efficiency, communication overhead, task allocation quality, robustness to agent failure, scalability, and social welfare. LLM-agent evaluation considers task success, reasoning quality, tool-use accuracy, factuality, cost, latency, robustness, and safety. A unified system must be evaluated across both sets of criteria. A system that produces elegant dialogue but fails execution is inadequate; a system that executes reliably but cannot interpret natural-language goals may also fail the intended use case.

Several benchmarks have shaped LLM-agent evaluation. AgentBench evaluates LLMs as agents across interactive tasks [64]. WebArena tests web navigation and task completion in realistic web environments [113]. ALFWorld aligns text instructions with embodied environments [92]. SWE-bench evaluates the ability to resolve real-world software issues [54]. GAIA emphasizes general assistant tasks requiring reasoning and tool use [69]. OSWorld evaluates computer-control agents in operating-system environments [108]. These benchmarks are useful, but they do not fully capture the problem of heterogeneous multi-agent coordination with symbolic verification.

The evaluation gap is therefore clear. Existing benchmarks evaluate individual LLM

Table 2.5: Evaluation dimensions for unified LLM-symbolic multi-agent systems.

Dimension	Representative questions
Task performance	Does the system achieve the specified goal, and with what cost, latency, and solution quality?
Coordination quality	Are tasks allocated effectively, messages grounded, conflicts resolved, and commitments tracked?
Symbolic validity	Do generated plans satisfy preconditions, constraints, and domain rules before execution?
Robustness	Does performance persist under rephrased goals, partial observability, agent failure, or adversarial inputs?
Safety and governance	Are tool calls authorized, logged, monitored, and constrained by explicit policies?

agents, web navigation, embodied tasks, coding tasks, or general assistant behavior. Classical MAS benchmarks evaluate coordination mechanisms under formal assumptions. Fewer evaluations measure whether LLM-mediated agents can coordinate through natural language while maintaining symbolic state, formal commitments, and verified execution. This gap motivates thesis-specific scenarios that test not only whether a final task is completed, but also whether the coordination trace is interpretable, valid, and safe.

A useful evaluation design should therefore include both outcome metrics and process metrics. Outcome metrics measure whether the task was completed, how long it took, how much it cost, and whether the output met quality criteria. Process metrics measure whether the correct agent accepted the task, whether the symbolic plan was valid, whether messages followed protocol, whether tool calls were authorized, and whether failures were detected at the proper boundary. This distinction is important because an LLM system can sometimes achieve a correct result through an unsafe or non-generalizable process. Scientific evaluation should avoid rewarding such accidental success without qualification.

Evaluation should also include ablations. A unified architecture can be compared against variants that remove the symbolic planner, remove the LLM interpreter, remove protocol tracking, or remove runtime monitoring. Such comparisons make it possible to test whether each component contributes to performance, safety, or interpretability. Without ablation, the architecture may appear effective only because the task is easy or because a single model performs most of the work. The state of the art in agent evaluation increasingly recognizes this issue, but unified LLM-symbolic MAS evaluation remains underdeveloped.

2.12 Synthesis and Research Gap

The reviewed literature shows that the components needed for unified coordination already exist, but they remain insufficiently integrated. Classical agent theory defines autonomy,

interaction, commitment, planning, and verification. MAS research provides mechanisms for task allocation, negotiation, shared-state coordination, norms, and distributed constraint reasoning. Symbolic AI supplies formal representations for knowledge, plans, rules, and control. LLM research supplies linguistic interpretation, flexible communication, tool-use scaffolding, and broad task generalization. Modern interoperability protocols supply practical interfaces for tools, context, and cross-agent communication.

Table 2.6: Synthesis matrix deriving requirements for the proposed framework.

Research line	Contribution	Limitation	Derived thesis requirement
Classical agents and MAS	Autonomy, roles, task allocation, negotiation, commitments, and coordination failures.	Does not by itself handle open-ended natural-language intent or modern LLM tool scaffolds.	Treat agents as accountable participants with explicit state, capabilities, and commitments.
ACLs and interaction protocols	Performatives, protocol state, content languages, and observable communicative force.	Structured acts are often absent from free-form LLM dialogue.	Map language-mediated exchanges to typed requests, acceptances, reports, and completion evidence.
Symbolic AI	Plans, constraints, schemas, rules, and verification conditions.	Only reasons over modeled facts, actions, and contracts.	Use symbols for authorization, compatibility, evidence, and refusal rather than for all semantic interpretation.
Neuro-symbolic and LLM planning	Translation between natural language, tools, formal goals, and external solvers.	LLM-generated plans can remain plausible but invalid under action semantics.	Separate candidate proposal from feasibility checking and executable commitment.
LLM-MAS frameworks	Role-based decomposition, critique, tool use, and flexible collaboration.	Prompted roles and conversation traces do not guarantee role semantics or accountability.	Require explicit ownership, authority, traceability, and process metrics beyond final output.

Research line	Contribution	Limitation	Derived thesis requirement
Modern interoperability protocols	Tool/context access, agent discovery, task exchange, and cross-framework connectivity.	Connectivity is not coordination, and discovery is not trust.	Keep protocol transport subordinate to coordination semantics, trust decisions, and state models.
Safety and evaluation	Monitoring, authorization, audit logs, benchmark practice, and process/outcome separation.	Full verification of open-ended LLM behavior remains impractical.	Verify the symbolic envelope: contracts, permissions, transitions, traces, and bounded auxiliary capabilities.

The state of the art nevertheless leaves several unresolved issues. First, the term agent is used inconsistently across classical MAS and LLM literature, which obscures the distinction between a language model, a tool-using application, and an autonomous system. Second, LLM-based coordination often relies on free-form conversation without explicit performatives, commitments, or protocol state. Third, symbolic planning and verification are often treated as optional tools rather than as structural components of safe execution. Fourth, emerging protocols improve connectivity but do not, by themselves, solve the semantic problem of coordination. Finally, benchmark practice has not yet converged on evaluation methods for systems that combine linguistic agents, symbolic agents, tool agents, and autonomous execution components.

These gaps define the thesis positioning. A unified coordination framework should treat LLMs as linguistic reasoning modules embedded inside full autonomous agents. It should translate between natural-language interaction and symbolic coordination structures. It should distinguish proposals from commitments, and commitments from executed actions. It should expose tools through controlled interfaces, represent shared state explicitly, and validate plans before execution. Such a framework does not reject the flexibility of LLMs; rather, it makes that flexibility usable in autonomous systems where coordination, accountability, and safety matter.

The resulting research direction is not a competition between neural and symbolic AI. It is an architectural question about how heterogeneous reasoning components should cooperate. LLMs are valuable because they make autonomous systems more communicative, adaptive, and accessible to human intent. Symbolic AI is valuable because it makes plans, constraints, commitments, and verification conditions explicit. MAS research is valuable because it explains how autonomous entities interact when no single component has all

information or authority. The contribution of a unified framework is to place these elements in a coherent coordination loop, where language supports interpretation, symbols support execution semantics, and protocols support collective behavior.

Chapter 3

Research Methodology

3.1 Study Design

The thesis follows design science: it constructs a protocol-adaptable coordination boundary and evaluates whether the implementation exhibits the declared authorization, refusal, execution, and recovery behavior. The unit of analysis is one coordination attempt, comprising the natural-language request, admitted registry snapshot, typed request, non-authoritative proposal, symbolic predicates, authorization decision, task attempts, artifacts, and persisted trace. Outcome correctness and process correctness are analyzed separately.

The evaluation is staged. Unit and integration tests exercise individual contracts and transitions. Seven deterministic cases exercise the complete in-process workflow. Container harnesses exercise A2A admission and dispatch across processes and lease-backed recovery through PostgreSQL. The main comparative study uses a versioned corpus of 36 cases and local LLM proposals. Results from an earlier implementation are classified as legacy and cannot supply numerical claims for version 0.2.

3.2 Versioned and Blinded Corpus

The version 0.2 corpus contains 24 base cases, four in each of six strata: direct feasible execution; decomposed feasible execution; feasible planning followed by runtime failure; missing capability or auxiliary overreach; schema, modality, graph, or ordering invalidity; and authority, typed-constraint, availability, or registry invalidity. Twelve controlled variations add four paraphrases, four availability changes, and four capability-alias perturbations. Public requests, registry snapshots, payloads, and shared proposals are stored separately from hidden reference labels.

The corpus generator computes a SHA-256 digest over public cases and hidden labels. A tutor or other qualified independent reviewer must approve the labels and minimum

justifications before scoring. The signed record identifies reviewer role, review date, and corpus hash. Reference labels are never included in model prompts. If the corpus changes after sign-off, generation fails rather than silently reusing the old approval.

3.3 Configurations and Fairness

Each applicable raw proposal is evaluated under: the full hybrid coordinator; a natural-language rule matcher; a structured-request rule-only oracle, explicitly interpreted as an upper bound; an LLM-only tool-aware baseline without a plan-level symbolic gate; and six hybrid ablations removing typed constraints, authority, contract validation, dependency gating, bounded auxiliary rules, or durable trace recording. Configurations receive the same request, registry snapshot, payload, and raw linguistic output where applicable. A baseline is not granted additional agents, credentials, hidden labels, or privileged registry information.

Planning-only decisions and runtime compliance are not conflated. Dependency-gating and durable-trace ablations may leave the planning decision unchanged while altering execution safety or observability. Those mechanisms are therefore scored through dispatch, skip, duplicate-effect, and trace-completeness measures rather than through feasibility accuracy alone.

3.4 Local Model Protocol

The fixed local models are qwen/qwen3-1.7b, google/gemma-4-e2b, and qwen/qwen3-8b. Qwen 14B is excluded because its installed nine-gigabyte quantization does not leave a reproducible operating margin on the 16 GB host. LM Studio is addressed at <http://127.0.0.1:1234/v1> with a non-secret local API-key value. Temperature is zero and seeds are 11, 22, 33, 44, and 55.

All other models are unloaded before a batch. The runner loads one exact identifier, verifies the model catalog, sends a sentinel request, and then runs all 36 cases for each seed before changing models. The prompt requests a typed request and between one and three typed candidate plans. Every response preserves the prompt, raw response, parsed object, returned model identifier, token metadata, latency, seed, and decoding parameters. This yields 540 immutable model outputs. Repeated identical outputs remain in the evidence set.

3.5 Metrics and Statistical Analysis

Decision measures include accuracy, precision, recall, F1, false acceptance, false refusal, status agreement, and category-level errors. Boundary measures include constraint, authority,

schema, modality, dependency, and evidence rejection; artifact-contract validity; required-artifact completion; authorization-before-dispatch; dependent-task skipping; capability-ID resolution; unresolved references; and trace completeness and isolation. Efficiency measures include messages, planner calls, tokens, median and interquartile latency, and coordination overhead. Recovery measures include duplicate dispatch, repeated effect, stale-write rejection, recovery latency, and terminal correctness.

Results are stratified by model and seed. Exact paired McNemar tests compare decision errors. Difference in proportions reports effect magnitude, and fixed-seed bootstrap intervals report 95% uncertainty for accuracy. No general superiority claim is made when the designed sample or statistical assumptions do not justify it.

3.6 Provenance and Reproducibility

Each run directory records Git SHA, dirty-state flag, corpus hash, model ID, prompt/schema version, seed, dependency-lock hash, Docker image digest, Python, PostgreSQL, A2A and LM Studio versions, operating system, CPU, RAM, and available accelerator information. Reports are written to new run-ID directories; convenience summaries are derived and may be regenerated, but the only copy of a raw run is never overwritten.

The release gates require all unit and integration tests, static lint and type checks, the seven deterministic cases, eleven Docker A2A checks, five distributed checks, the third-party A2A check, evidence-schema checks, and LuaLaTeX/BibTeX compilation. Coverage thresholds are 85% overall, 90% for feasibility, coordination execution, and trace code, and 80% for the PostgreSQL store and evaluation runners. A release cannot be frozen while any threshold, independent sign-off, institutional metadata, or final-commit field is open.

3.7 Validity Boundaries

The corpus is designed rather than sampled from a deployment population. The agents are mostly controlled fixtures, and the local models are small. Contract validity does not establish semantic truth. Lease and fencing evidence concerns measured crash faults, not consensus or Byzantine tolerance. A2A admission establishes local origin, transport, trust, and credential policy; it does not establish cryptographic identity or malicious-agent resistance. These boundaries constrain every answer to the research questions.

Chapter 4

Proposed Theoretical Framework

This chapter develops the theoretical framework proposed by the thesis. The framework addresses a specific coordination problem: how a distributed collection of heterogeneous agents can accept a problem stated in natural language, establish whether the problem lies within its current operational boundary, and construct a coordinated solution without allowing linguistic plausibility to substitute for executable justification. The proposed answer combines a central coordinating authority with distributed execution, capability-based admission, explicit commitments, and a symbolic feasibility decision. Language-oriented components interpret and propose; symbolic components authorize, record, and reject when the available evidence is insufficient.

The framework is intentionally independent of a particular communication protocol, model provider, programming language, or deployment platform. Chapter 5 instantiates the concepts using concrete technologies, but those choices are not part of the theoretical claim. The present chapter instead defines the objects, boundaries, and properties that any conforming implementation must preserve.

4.1 Purpose and Contribution

Open multi-agent systems derive value from heterogeneity. Agents can differ in ownership, internal architecture, reasoning method, data access, and operational authority while still contributing to a shared objective. The same heterogeneity also complicates coordination. A fluent description of a capability does not establish that the capability exists; a plausible decomposition does not establish that its subtasks can exchange compatible artifacts; and a successful delegation message does not establish that the delegated work has been completed. These distinctions are especially important when large language models participate in interpretation and planning, because linguistic confidence can conceal missing capabilities or violated constraints.

The framework treats this problem as one of *controlled openness*. Participation is open

in the sense that independently implemented agents can be admitted through externally visible contracts. It is controlled because admission, authority, capability coverage, and plan acceptance remain explicit decisions. The operational boundary is therefore neither the set of tasks an LLM can describe nor the set of outcomes that appear plausible in conversation. It is the set of task assignments that can be justified from the currently admitted agents, approved auxiliary capabilities, compatible data flows, and applicable authority constraints.

The principal contribution is a disciplined boundary between linguistic proposal and symbolic authorization. Natural-language reasoning is used where ambiguity and semantic comparison make it valuable: interpreting a request, comparing capability descriptions, and proposing a decomposition. Symbolic reasoning is used where explicit structure is available: checking assignments, schemas, modalities, dependencies, trust constraints, completion evidence, and lifecycle rules. The result of coordination is total at the planning boundary: either an accepted plan is produced with an auditable justification, or a structured infeasibility report identifies why no candidate plan can presently be authorized.

Figure 4.1 summarizes this separation. The central coordinating authority is a logical decision role rather than a requirement that all execution occur in one process. Participating agents remain distributed and retain control of their internal operation. Centralization applies to the acceptance of the shared plan and the maintenance of its coordination state. A fault-tolerant implementation may therefore replicate coordinator processes or move state into durable storage, provided that each coordination session still has one authoritative plan state and one accepted authorization decision.

4.2 Assumptions and Design Requirements

The framework rests on five assumptions. First, participating agents are treated as black boxes with respect to their internal reasoning. An agent may use rules, a planner, a language model, retrieval, statistical learning, deterministic software, or a hybrid of these methods. Coordination depends only on the contract the agent exposes and on the evidence it returns. Second, the registry is evaluated as a session-level snapshot. Agents may join, leave, or change between sessions, but a feasibility decision is made against a definite set of available descriptions and states.

Third, capability descriptions and admission metadata are assumed to be sufficiently honest for contractual reasoning. This is not an assumption that agents are infallible. It means that the coordinator may use an admitted capability claim as evidence when constructing a plan. Misrepresentation, compromised identities, and adversarial capability advertisements require trust-establishment mechanisms beyond the present theory. Fourth, the central coordinator is trusted to apply the declared checks correctly. The framework constrains this role through explicit inputs, outputs, and traces, but it does not solve Byzantine coordination

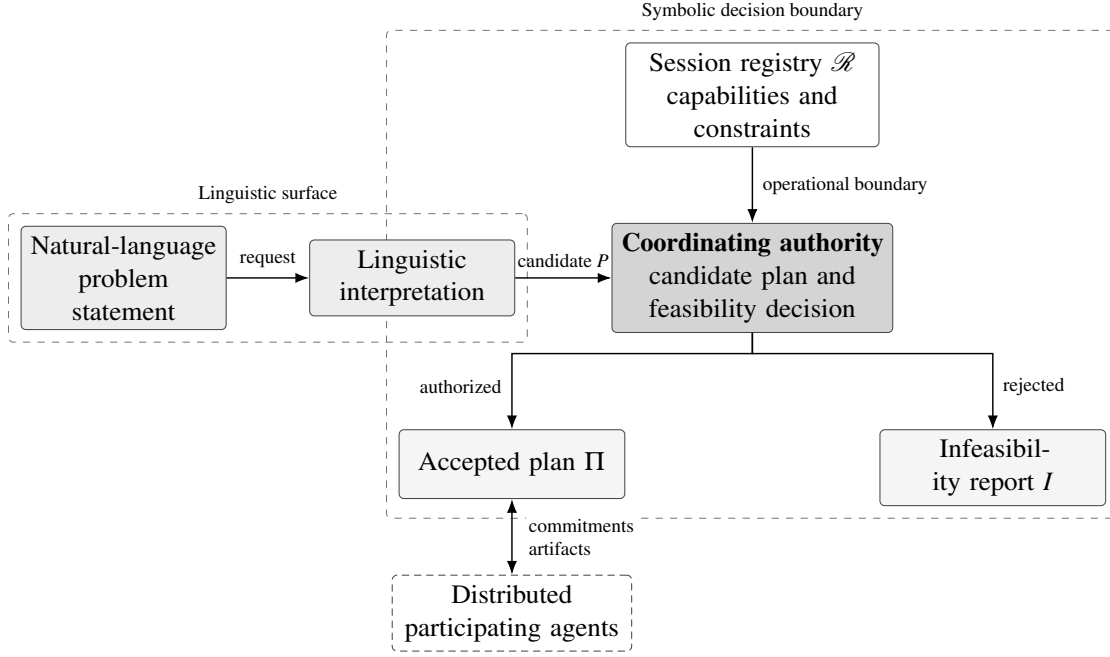


Figure 4.1: Conceptual boundary of the proposed framework. Linguistic interpretation produces a candidate problem representation; the coordinating authority evaluates it against the session registry and returns either an authorized plan or an explicit infeasibility report. Execution remains distributed among participating agents.

among competing coordinators. Fifth, deterministic validation is limited to properties that can be expressed in a contract. Schema validity, provenance, bounds, permissions, and required fields may be decidable even when the semantic quality of an open-ended result is not.

Within those assumptions, the design is governed by nine requirements. *Openness* permits independently developed agents to enter or leave through explicit admission procedures. *Vendor independence* prevents coordination semantics from depending on an agent’s internal framework or model provider. *Interoperability* requires shared representations for capabilities, tasks, status, artifacts, and errors. *Distributed operation* keeps participating agents independently deployable and avoids shared hidden state as a prerequisite for cooperation. *Coordinated authority* assigns responsibility for the global feasibility decision and the accepted plan. *Simplicity* prefers direct delegation over decomposition and deterministic checks over discretionary linguistic judgments when both can decide the same condition. *Observability* records decisions, commitments, execution events, and artifacts. *Fault containment* treats timeouts, unavailable agents, invalid artifacts, and rejected plans as explicit states rather than exceptional conversational endings. Finally, *bounded authority* prevents either the coordinator or a generated auxiliary capability from acquiring permissions merely because a request would benefit from them.

These requirements create a deliberate asymmetry. Execution is distributed, but authorization of the shared plan is centralized. The asymmetry is not asserted as a universal necessity

for all multi-agent systems; decentralized negotiation and consensus are valid alternatives. It is a design choice for this framework because a single decision role provides a definite answer to the question the thesis studies: whether a requested solution is justified within the current system boundary.

The scope is correspondingly limited. The framework does not prove the correctness of an agent's internal reasoning, guarantee that an accepted plan will succeed at runtime, optimize resource allocation in competitive markets, or provide decentralized consensus. It also does not replace an agent communication language or transport protocol. Its claim is narrower: under the stated assumptions, it makes the basis for plan acceptance, delegation, and refusal explicit and testable.

4.3 Conceptual Model of Agents and Capabilities

The conceptual model separates an agent's implementation from its externally relevant coordination state. This permits a language-model agent, a symbolic planner, a database service, and a deterministic program to participate under the same abstraction without implying that their internal forms of autonomy are identical.

Definition 4.1 (Agent). An agent is a tuple

$$a = (id, C(a), M_{in}(a), M_{out}(a), \tau(a), \sigma(a)),$$

where id is a unique identity, $C(a)$ is a finite set of declared capabilities, $M_{in}(a)$ and $M_{out}(a)$ are accepted input and produced output modalities, $\tau(a)$ is an admitted trust or authority level drawn from a partial order, and $\sigma(a)$ is the agent's observable availability state for the current session.

Identity and availability are included because capability alone is insufficient for delegation. An agent that advertises the correct transformation but is unavailable, unauthenticated, or unauthorized for the requested context cannot justify a feasible assignment. The trust value $\tau(a)$ is intentionally abstract: an implementation may derive it from credentials, local policy, organizational membership, or another admission mechanism.

Definition 4.2 (Capability contract). A capability contract is a tuple

$$c = (name, \Sigma_{in}, \Sigma_{out}, \gamma, e, v),$$

where Σ_{in} and Σ_{out} are input and output schemas, γ is a set of preconditions and authority constraints, e specifies the evidence expected on completion, and v is a deterministic contract validator for that evidence and the produced artifact.

The validator v establishes conformance to the declared contract, not universal semantic correctness. For a numerical service, it may recompute a result; for a document generator, it may only verify format, provenance, required sections, and declared bounds. This distinction prevents the theory from claiming that every open-ended capability has a complete oracle. Where semantic correctness is undecidable or expensive, the contract must state the evidence that can actually be checked and leave residual uncertainty visible.

Capability matching is represented by the relation $a \models q$, meaning that at least one capability in $C(a)$ covers requirement descriptor q under the applicable schemas, modalities, and constraints. Coverage may be established by exact symbolic matching or proposed by semantic comparison and then confirmed against explicit contract fields. A linguistic similarity score alone is not sufficient to establish $a \models q$.

Definition 4.3 (Registry and operational boundary). Let $\mathcal{R}$ be the registry snapshot for a coordination session. Its capability space is

$$K(\mathcal{R}) = \bigcup_{a \in \mathcal{R}} C(a),$$

and its supported modalities are

$$M(\mathcal{R}) = \bigcup_{a \in \mathcal{R}} (M_{\text{in}}(a) \cup M_{\text{out}}(a)).$$

The operational boundary $\mathcal{B}(\mathcal{R})$ is the set of agent–capability pairs (a, c) for which $a \in \mathcal{R}$, $\sigma(a)$ is available, $c \in C(a)$, and the admission and authority constraints applicable to c are satisfied.

The boundary is evaluated rather than presumed. It may contract when an agent becomes unavailable or loses authority, and it may expand when a new agent is admitted. Task-local auxiliary capabilities can extend it only under the bounded rules defined later in this chapter. The boundary therefore describes what the system can justify at a particular decision point, not what it might be able to learn or deploy in the future.

4.4 Linguistic Agents and Symbolic Mediation

A linguistic agent is an agent whose principal capabilities transform, interpret, or produce language-mediated artifacts. Typical capabilities include intent interpretation, summarization, classification, translation, information extraction, semantic matching, question answering, and natural-language explanation. Such agents are necessary because user goals and capability descriptions frequently arrive as prose, whereas coordination requires structured task and artifact contracts.

Definition 4.4 (Linguistic agent). An agent a is linguistic with respect to a capability $c \in C(a)$ when the contract for c accepts or produces natural language and the capability’s primary function is to mediate between linguistic meaning and another linguistic or structured representation.

This definition concerns function rather than implementation. A linguistic agent may use a large language model, a smaller statistical model, rules, retrieval, a controlled vocabulary, or a hybrid pipeline. Broad contextual interpretation may justify an LLM, whereas language detection, constrained extraction, or normalization may be implemented more simply. Treating every language operation as an LLM task would weaken the simplicity requirement and could increase cost, latency, and uncertainty without adding relevant capability.

The linguistic–symbolic bridge is consequently modeled as a sequence of artifacts and validation boundaries, not as a single translation component. The theory requires that language-mediated interpretation, candidate proposal, symbolic authorization, and feedback grounding remain distinguishable even when a concrete implementation realizes them through the same software runtime. Four transitions define the core loop.

Transition T1: interpretation. A user expression u is transformed into a candidate problem representation $\hat{P}$. Because interpretation may be uncertain, $\hat{P}$ is not yet authoritative. Required fields, explicit constraints, ambiguities, and referenced artifacts are checked before it is admitted as P .

Transition T2: proposal. Given the admitted problem and $\mathcal{B}(\mathcal{R})$, a linguistic reasoner proposes a candidate decomposition $\hat{\Pi}$. This stage may use semantic matching to compare the goal with natural-language capability descriptions. It proposes assignments but does not authorize them.

Transition T3: authorization. Symbolic predicates evaluate $\hat{\Pi}$ against capability coverage, data-flow compatibility, dependencies, availability, authority, and evidence contracts. The result is an accepted plan Π or an infeasibility report I .

Transition T4: feedback grounding. Status updates, artifacts, timeouts, invalid outputs, and protocol errors become structured observations linked to the corresponding task and commitment. These observations may discharge a commitment, trigger bounded replanning, or terminate the plan with an execution failure.

Between T2 and T3, semantic names are normalized only through deterministic registry evidence. A model may propose a phrase such as “make the brief” or may refer to an agent identifier rather than the internal capability name. That proposal can be admitted as a symbolic requirement only if it resolves to exactly one advertised capability through an explicit rule, such as an exact normalized capability name, a registered agent identifier with a

single advertised skill, or another deterministic registry mapping. Ambiguous or unresolved names remain proposal evidence and cause refusal or a request for a clearer boundary; they do not become hidden capabilities.

Natural-language explanations may be rendered from either Π or I , but the explanation is not the authoritative state. The symbolic artifact remains available for inspection and execution. Figure 4.2 shows the flow across the three conceptual lanes.

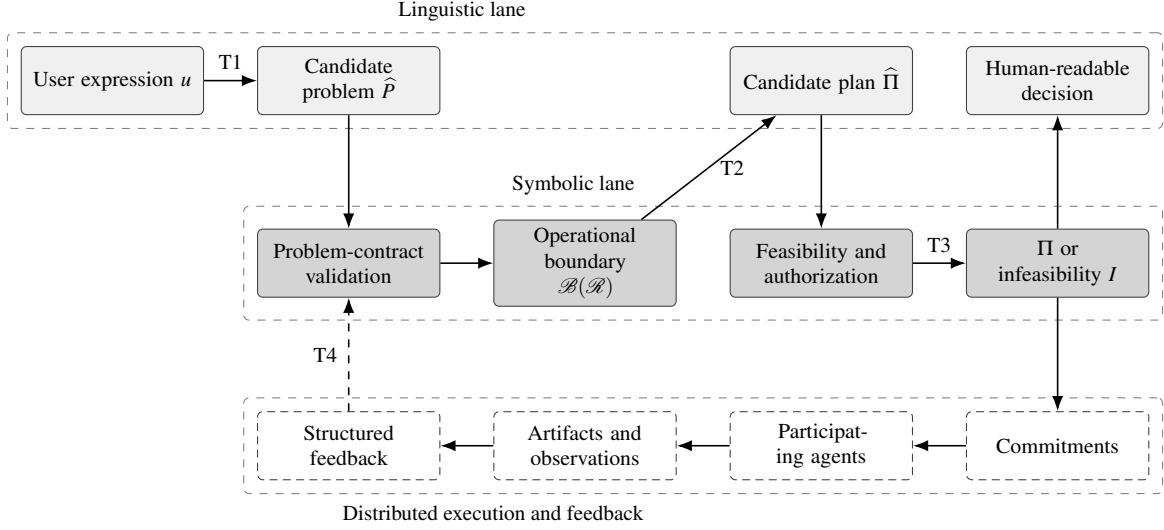


Figure 4.2: Artifact flow across the linguistic, symbolic, and distributed-execution lanes. Linguistic components produce candidate representations; symbolic validation authorizes a plan or records infeasibility; execution observations return through a structured feedback path.

The framework is compatible with, but not equivalent to, classical BDI terminology. The admitted problem contributes to the coordinator’s beliefs, a candidate plan represents a proposed course of action, and authorization establishes an intention-like commitment to execution [82, 83]. This is an architectural analogy rather than a claim that the tuples defined here constitute a complete BDI semantics. Similarly, structured delegations and status reports can carry communicative forces related to request, inform, and commit [5, 88, 16, 30], but their exact message encoding belongs to the implementation layer.

4.5 Bounded Auxiliary Linguistic Capabilities

An admitted registry will not contain every narrow language transformation that a decomposition may require. Rejecting an otherwise feasible plan because a trivial normalization or extraction step is absent can be disproportionate; assigning the same step to a general-purpose LLM can also be unnecessarily complex. The framework therefore permits a task-local auxiliary linguistic capability when the missing operation can be constructed from an approved, bounded method and validated against an explicit contract.

Auxiliary synthesis does not mean unrestricted generation of executable agents, arbitrary code, or newly trained models. It means constructing a temporary capability specification by selecting or composing methods from an admissible family and binding their parameters to the current task. The resulting logical agent may be realized by an existing sandbox, rule engine, model endpoint, or controlled execution service in the implementation. The theory requires the specification and its bounds; it does not prescribe the runtime that realizes them.

Definition 4.5 (Auxiliary capability specification). An auxiliary capability specification is a tuple

$$x = (\textit{purpose}, \Sigma_{\text{in}}, \Sigma_{\text{out}}, m, v, b, \lambda),$$

where m is a selected method from an approved family, v is its deterministic contract validator, b states resource and authority bounds, and λ binds the specification to the lifecycle of one coordination plan.

Four rules govern admission of an auxiliary specification.

Rule A1 (Approved construction). The method m must be selected from, or composed entirely from, an approved finite family whose operational semantics are known. Synthesis may bind parameters and connect approved steps; it may not introduce arbitrary executable code or an unreviewed model.

Rule A2 (Contract validation). The input schema, output schema, expected evidence, and deterministic validator v must be available before the auxiliary capability is authorized. Failure of v is an execution failure, not a linguistically negotiable result.

Rule A3 (Bounded effects and authority). The capability must be side-effect free or restricted to effects explicitly authorized by b . It may not acquire persistent write access, external credentials, or privileges that are absent from the accepted plan.

Rule A4 (Lifecycle and provenance). The lifecycle token λ limits the auxiliary capability to its plan, and every invocation records the selected method, parameters, inputs, outputs, and validation result. The specification does not enter the persistent registry automatically.

These rules distinguish a bounded micro-capability from an attempt to synthesize expertise. Extracting fields from a fixed schema, normalizing a constrained identifier, or applying an approved classifier may be eligible. Producing a medical diagnosis, forecasting an unconstrained market, or generating privileged database operations is not eligible merely because a language model can describe a procedure. When no registered capability and no admissible auxiliary specification covers a requirement, the correct planning result is infeasibility.

4.6 Coordination, Commitments, and Communication Abstraction

The framework uses hierarchical coordination in a limited sense. The coordinator owns interpretation state, the registry snapshot, candidate-plan evaluation, and acceptance of the joint plan. Participating agents retain autonomy over whether and how they execute a delegated capability, subject to their advertised contracts. The hierarchy therefore establishes responsibility for the global decision without converting distributed agents into internal modules of the coordinator.

Central coordination is selected here because local views are insufficient to decide joint feasibility. An individual agent may know that it can perform one transformation without knowing whether another agent can consume its output or whether the complete artifact set satisfies the user's goal. The coordinator maintains the cross-agent dependency graph and the evidence required to justify the overall proposal. This choice also introduces a concentration of trust and a potential availability bottleneck, which are acknowledged as limitations rather than presented as universal properties of open systems. Crash fault tolerance can be added without changing the theoretical role of the coordinator by recording the session state durably and allowing another coordinator instance to resume the same logical role after failure. Byzantine coordination among mutually suspicious or competing coordinators remains outside this framework.

Definition 4.6 (Commitment). A commitment is a tuple

$$\mu = (d, r, t, e, \kappa_t, s),$$

where d is the debtor agent, r is the coordinating creditor acting on behalf of the requester, t is the delegated task, e is the expected completion evidence, κ_t is the discharge condition, and s is the observable commitment state.

A commitment is created only after plan authorization and delegation acceptance. It is discharged when the expected artifact and evidence satisfy the task contract, or terminated with a recorded failure. Maintaining commitments as symbolic state addresses a weakness of conversation-only coordination: dialogue can indicate willingness, but it does not by itself establish who owes which artifact or what observation counts as completion [93].

4.6.1 Required Communication Properties

The communication layer transports coordination artifacts but does not decide their meaning. To preserve the theory, an implementation must provide authenticated addressability, correlated task delegation, capability and contract conveyance, status and artifact delivery, timeout

and cancellation semantics, and detectable failures. Messages associated with a session must carry enough identity and correlation information to reconstruct their causal order even when the underlying transport does not guarantee global ordering.

Capability descriptions must be versioned or otherwise bound to the registry snapshot used for feasibility. Delegations must identify the task, plan, expected artifact, and applicable lifecycle. Results must identify their producing agent and task so that validation and commitment discharge are unambiguous. Duplicate, delayed, or out-of-order messages must be detectable and handled idempotently at the coordination layer. Lack of progress must become an observable timeout rather than an assumption that the agent remains active.

These requirements define a communication abstraction, not a specific protocol. Any protocol or composition of protocols may instantiate it if the required artifacts and observations can be represented without exposing agent internals. The abstraction may be implemented with different transports, local runtimes, or linguistic runtimes, provided that each exposes the same capability, task, artifact, and trace contracts to the feasibility model. Planning, feasibility, and authorization remain coordination-layer responsibilities rather than transport-layer responsibilities. Substitutability is therefore conditional: changing the transport or runtime must not change the semantics of capability admission, task correlation, commitment state, or feasibility evidence.

4.7 Feasibility Model

Feasibility is a pre-execution judgment about whether a plan is justified by the current contracts and constraints. It is not a prediction that every agent will execute successfully. Runtime failure remains possible and is handled through commitment state and structured feedback. This separation is essential: without it, a failed execution could be mistaken for an unsound planning decision, or an optimistic planning decision could be defended by a later successful accident.

Definition 4.7 (Problem request). A problem request is a tuple

$$P = (g, \mathcal{Q}, \chi, \mathcal{O}, \Gamma),$$

where g is the goal, $\mathcal{Q}$ is a finite set of required capability descriptors, χ is contextual information, $\mathcal{O}$ is the required artifact set, and Γ is the set of explicit constraints. A descriptor in $\mathcal{Q}$ may be marked as auxiliary-eligible, but eligibility is confirmed only by Rules A1–A4.

Definition 4.8 (Coordination plan). A coordination plan is a tuple

$$\Pi = (\mathcal{T}, q, \alpha, \prec, \mathcal{X}_{\Pi}, \mathcal{A}_{\text{out}}, \kappa),$$

where $\mathcal{T}$ is a finite set of tasks, $q : \mathcal{T} \rightarrow \mathcal{Q}$ maps each task to its capability requirement, α assigns each task to a registered agent or an approved auxiliary specification, $\prec$ is a dependency relation, $\mathcal{X}_\Pi$ is the finite set of auxiliary specifications, $\mathcal{A}_{\text{out}}$ is the expected artifact set, and κ is the plan completion criterion.

For an auxiliary specification x , the notation $x \models q(t)$ means that its approved method and contract cover the requirement of task t . For a registered agent, coverage retains the meaning established earlier. The executor assignment is total when every task has exactly one accountable executor at authorization time; redundancy or collaborative subtasks can be represented explicitly as additional tasks rather than as an ambiguous assignment.

Definition 4.9 (Feasibility). Given a request P and registry snapshot $\mathcal{R}$, feasibility holds when there exists a plan Π such that

$$\text{Feasible}(P, \mathcal{R}) \iff \exists \Pi [\text{Complete}(\Pi, P) \wedge \text{Covered}(\Pi, \mathcal{R}) \wedge \text{Compatible}(\Pi) \\ \wedge \text{Acyclic}(\prec) \wedge \text{Authorized}(\Pi, \mathcal{R}) \wedge \text{Verifiable}(\Pi)].$$

These predicates are operational acceptance criteria for implementation and evaluation, not a proof of the internal correctness of arbitrary agents. They specify which evidence must be present in the coordinator's trace before a plan may be authorized. The claim is therefore limited to pre-execution justification under the admitted contracts and registry snapshot.

The predicates make the acceptance criteria explicit. Complete requires that the task set and expected artifacts cover $\mathcal{Q}$ and $\mathcal{O}$. Covered requires

$$\forall t \in \mathcal{T}, \quad \alpha(t) \models q(t),$$

with every registered assignment drawn from $\mathcal{B}(\mathcal{R})$ and justified by a recorded capability contract, and every auxiliary assignment drawn from $\mathcal{X}_\Pi$ after satisfying Rules A1–A4. Compatible requires each dependency edge to connect recorded output and input schemas, modalities, and artifact identifiers, or an explicitly admitted bridge with its own validator. Acyclic requires $\prec$ to define a valid finite partial order. Authorized checks availability, trust, permissions, resource limits, and the constraints in Γ against the registry snapshot and policy evidence used for the decision. Verifiable requires a completion-evidence contract and deterministic validator for every task and requires κ to be decidable from recorded task states, artifacts, and validation results.

An infeasibility report is a structured artifact $I = (\text{code}, U, E, H)$, where U identifies unsatisfied requirements or constraints, E records the checks that produced the verdict, and H contains admissible remediation hints, such as admitting a missing capability or providing a required modality. A remediation hint is not a fabricated plan. It explains how the boundary would need to change before the request could be reconsidered.

The operational boundary is dynamic across sessions but fixed for a single authorization decision. If the registry changes during planning, the candidate must be re-evaluated against the new snapshot. If it changes after authorization, the affected commitment becomes unavailable or invalid and enters the feedback loop. This rule avoids accepting a plan from a capability view that no longer corresponds to the agents available for execution.

4.8 Properties of the Framework

The following properties state the behavior expected from a conforming implementation. They are conditional on the assumptions and contract descriptions introduced above; they are not claims about the internal correctness of arbitrary agents.

Property 4.1 (Justified plan acceptance). If the coordinator accepts Π , every task has an accountable executor whose admitted contract covers its requirement, every dependency is compatible, and every explicit authority and validation condition used by Definition 4.9 has passed. Acceptance does not imply that execution cannot subsequently fail.

Property 4.2 (Simplicity preference). If one admitted agent can satisfy the complete request under the same constraints and evidence requirements, the coordinator selects direct delegation before considering decomposition. Among evaluated multi-agent candidates, it prefers fewer tasks, fewer auxiliary specifications, and a shallower dependency graph, in that order, unless an explicit request constraint establishes another priority.

Property 4.3 (Bounded auxiliary synthesis). For every accepted plan, $\mathcal{X}_\Pi$ is finite; each member satisfies Rules A1–A4; and no member remains valid after its lifecycle token expires.

Property 4.4 (Traceability). Every decision can be reconstructed from the request representation, registry snapshot, candidate plan, predicate results, selected assignments, commitments, validation evidence, and execution observations. The trace distinguishes planning infeasibility, coordination failure, agent failure, and artifact-validation failure.

Property 4.5 (Explicit refusal). At the planning boundary, the coordinator returns either an accepted plan or an infeasibility report. Missing coverage, authority, compatibility, or evidence cannot be replaced by an unsupported linguistic assertion of feasibility.

These properties provide evaluation targets rather than completed proofs. The implementation must operationalize each predicate, and the empirical evaluation must examine whether observed traces conform to the stated behavior.

4.9 Worked Example

Consider a registry snapshot containing three agents. Agent a_1 summarizes English documents and returns a textual summary with source references. Agent a_2 computes arithmetic expressions over typed numeric inputs and returns both a value and a reproducible calculation trace. Agent a_3 executes read-only aggregate queries expressed through a constrained query object and returns a tabular artifact. All three agents are available and admitted for the relevant data.

The user requests: “Summarize the quarterly report, calculate the percentage change in revenue from Q1 to Q2, and list the five customers with the highest total order value.” Interpretation produces a candidate request whose required capabilities are document summarization, extraction of two named quarterly values, percentage calculation, and constrained aggregation. The required artifacts are a summary, a percentage, and a customer table.

Direct delegation fails because no single agent covers the complete request. The summarization, arithmetic, and aggregation requirements are covered by a_1 , a_2 , and a_3 , respectively. The registry lacks the narrow transformation that extracts the Q1 and Q2 revenue cells from a table with a known schema. This gap is auxiliary-eligible: an approved column-selection method can be parameterized with the two quarter labels, its output schema is a pair of currency values, and its validator checks column identity, numeric type, provenance, and cardinality. The coordinator therefore proposes the task-local specification x_1 rather than inventing a general table-understanding agent.

The candidate plan contains tasks t_1 for summarization, t_2 for percentage calculation, and t_3 for constrained aggregation, together with auxiliary specification x_1 . The report is supplied independently to t_1 and x_1 ; the validated revenue pair produced by x_1 is the input to t_2 ; and a typed aggregate request is supplied to t_3 . The outputs of t_1 , t_2 , and t_3 are collected into the required result. Figure 4.3 presents this plan as a task-and-artifact graph rather than as another architectural overview.

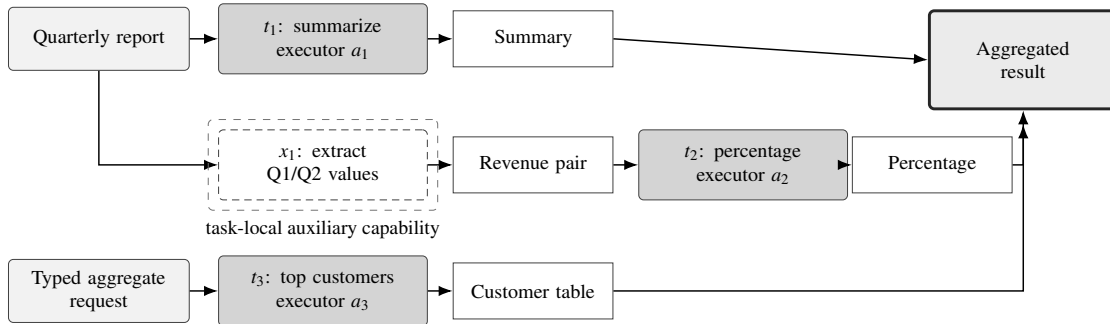


Figure 4.3: Task-and-artifact graph for the worked example. The temporary extractor x_1 performs only the bounded Q1/Q2 field selection; registered agents remain responsible for summarization, calculation, and constrained aggregation.

Feasibility holds because the task set covers the required capabilities and artifacts, each assignment is admitted, the extractor satisfies the auxiliary rules, the revenue-pair schema is compatible with the calculator input, the dependency graph is acyclic, and every task has a contract validator and completion condition. Authorization creates commitments for a_1 , a_2 , and a_3 and records the lifecycle of x_1 .

Now extend the request with “predict next year’s revenue.” No registered agent advertises a forecasting capability, and forecasting cannot be reduced to an approved bounded language transformation with deterministic contract validation. The coordinator therefore returns an infeasibility report identifying the uncovered forecasting requirement. It may state that admitting a forecasting agent would change the operational boundary, but it does not synthesize such expertise or allow the LLM’s narrative prediction to stand in for the missing capability.

The example illustrates the intended division of labor. Linguistic interpretation identifies the requested operations and proposes a decomposition. Symbolic authorization verifies the exact capability and artifact chain. Auxiliary synthesis is used only for a narrow, controlled transformation. Explicit refusal preserves the boundary when the missing capability is substantive.

4.10 Limitations and Chapter Synthesis

The framework has several limitations. Its contractual reasoning depends on the quality and honesty of admitted descriptions. A malicious or substantially inaccurate capability claim can invalidate the justification even if the coordinator applies the predicates correctly. Trust establishment, attestation, reputation, and adversarial admission are therefore complementary research problems. The session-snapshot model also simplifies highly dynamic environments; frequent changes may require repeated authorization and could reduce practical liveness.

The coordinator is a logical concentration of authority and state. This simplifies accountability and makes the feasibility decision explicit, but it creates a potential bottleneck and single point of failure if realized as one volatile process. The fault-tolerance path used by the implementation is crash resilience rather than Byzantine agreement: durable coordination state can preserve the request, registry snapshot, authorization decision, commitments, task observations, artifacts, leases, fencing tokens, and trace so that a standby coordinator can resume the same session. This preserves a unique plan state while avoiding dependence on one coordinator process. Stronger mechanisms for malicious coordinators, adversarial registries, or Byzantine behavior remain outside the present framework.

Deterministic validation has a deliberately limited meaning. A validator can confirm contract conformance and available evidence; it cannot guarantee the truth, usefulness, or safety of every semantically open-ended artifact. Human review, redundant agents, domain-

specific verification, or runtime monitoring may be required for high-impact tasks. Likewise, an accepted plan establishes pre-execution justification rather than guaranteed completion. Agents can fail, networks can partition, and external resources can change after authorization. The definitions and predicates are therefore an executable formalization of the proposed boundary, not a proof of global soundness, completeness, decidability limits, or computational complexity for arbitrary agent ecosystems.

Auxiliary capabilities are intentionally conservative. Restricting them to approved methods prevents the framework from treating code generation or model generation as an unbounded source of new competence, but it also means that some technically solvable requests will be rejected until a suitable method or agent is admitted. This is a purposeful tradeoff: the framework values a defensible boundary over maximal apparent coverage.

Within these limits, the contribution is a protocol-independent coordination model that combines distributed agents with central plan authorization, linguistic proposal, symbolic feasibility, explicit commitments, bounded auxiliary capabilities, and informative infeasibility. Chapter 5 maps these abstractions to a concrete system design.

Chapter 5

System Design: Coordination Agent over SDK Middleware

5.1 Instantiation of the Framework

This chapter instantiates the protocol-independent theory from Chapter 4 as a Python coordination agent built over a middleware SDK. The theoretical contribution does not require a particular transport; for the concrete prototype, the implementation selects the Agent2Agent (A2A) protocol as one interoperability adapter. A2A is an open protocol intended to support communication and coordination among agents built with different technologies. Its project documentation describes standardized communication, Agent Cards for discovery, flexible interaction patterns, and rich data exchange through text, files, and structured JSON [1]. Google’s announcement similarly frames A2A as an open protocol for secure information exchange and coordination among agents across enterprise platforms [38].

The implementation separates two roles. `CoordinationSdk` is the communication and runtime middleware. It registers remote A2A agents, local Python agents, and Lingo-backed linguistic agents; normalizes their capabilities; dispatches authorized tasks; converts artifacts; records runtime failures; and exposes communication traces. It does not plan, decide feasibility, authorize decompositions, or coordinate complete user requests.

`CoordinationAgent` is the central coordinating role. It receives the user request, uses the SDK to inspect the registry and capability index, invokes SDK-registered linguistic capabilities when interpretation or candidate planning is needed, builds direct or decomposed candidate plans, calls the symbolic `FeasibilityAnalyzer`, authorizes or refuses the plan, dispatches only authorized tasks through the SDK, and aggregates the final result. The agent never calls raw A2A transports, raw local handlers, or raw Lingo runtimes directly. All execution traffic still passes through `CoordinationSdk`.

A2A remains the default wire model for independently deployed agents. Remote agents

are admitted through A2A Agent Cards and invoked through A2A-compatible task exchange. Local Python agents and Lingo-based linguistic agents are registered through the SDK and normalized into the same internal agent records, with identity, capabilities, input and output modes, trust metadata, invocation endpoint, and validation contract. Local and linguistic agent authors are therefore not required to implement A2A plumbing, while the coordination agent can reason over a uniform registry.

The prototype also uses Lingo as an optional linguistic and context-engineering substrate [41]. Lingo is not hidden inside the SDK as a coordinator. Instead, Lingo-backed interpretation, extraction, classification, or candidate-planning behavior is exposed as SDK-registered linguistic capabilities. Non-linguistic local agents can use the SDK without depending on Lingo. The prototype dependency set pins `lingo-ai==2.0.2` and uses `a2a-sdk`, `pydantic`, `httpx`, `FastAPI`, `Uvicorn`, `pytest`, and `pytest-asyncio` for the executable implementation, service wrapper, and tests.

For deployment and reproducibility, the repository includes a thin HTTP service and Docker configuration around the same `CoordinationAgent` and `CoordinationSdk` objects. The service exposes health, registry, plan, coordinate, resume, and feasibility endpoints. It does not bypass the SDK boundary or change the theoretical ownership of planning and authorization. The implementation can run with the JSONL-backed local store used by simple tests, or with a PostgreSQL-backed shared coordination store selected through `COORDINATION_STORE_URL`. The PostgreSQL mode supports replicated coordinator processes by combining per-session leases, fencing tokens, idempotency keys, and durable terminal-result constraints.

The mapping between formal objects and implementation records is direct. The records defined in Section 5.4 materialize the formal objects of Chapter 4 as follows.

- `CoordinationAgent` materializes the central coordinating authority. It owns interpretation workflow, direct matching, decomposition, feasibility checks, auxiliary decisions, authorization, and final result construction.
- `CoordinationSdk` materializes the communication abstraction. It owns registry admission, A2A gateway behavior, local and Lingo runtimes, task dispatch, artifact normalization, runtime failure recording, and communication trace access.
- `CoordinationStore` materializes durable coordination state. Its JSONL adapter preserves the local testing backend; its PostgreSQL adapter stores sessions, events, leases, registry snapshots, plan generations, commitments, attempts, artifacts, and terminal results.
- `AgentRegistryEntry` materializes an agent a (Definition 4.1), supplying id , declared capabilities $C(a)$, input modes $M_{in}(a)$, output modes $M_{out}(a)$, local admission policy

$\tau(a)$, observable status $\sigma(a)$, and the invocation endpoint used by the SDK.

- `CapabilityRequirement` materializes a required capability descriptor $q \in \mathcal{Q}$ with modalities, schemas, constraints, trust level, and auxiliary eligibility.
- `ConstraintSpec` materializes one deterministic predicate with a stable identifier, declared evidence source, JSON-pointer path, typed operator, expected value, and optional requirement/task scope. Legacy prose is readable but unresolved and therefore cannot authorize.
- `ValidationContract` combines complete JSON Schema validation, required artifacts, kinds, fields, evidence types, and an optional registered validator. `CompletionContract` defines required terminal task states, final artifacts, and validator success.
- `ProblemRequest` materializes the request $P = (g, \mathcal{Q}, \chi, \mathcal{O}, \Gamma)$ from Definition 4.7.
- `SolutionProposal` materializes the coordination plan Π from Definition 4.8, including task requirements, assignments, dependencies, auxiliary specifications, expected artifacts, and completion criteria.
- `FeasibilityReport` records the verdict and predicate evidence from Definition 4.9. It is produced by the `FeasibilityAnalyzer` at the `CoordinationAgent` layer.
- `GeneratedNlpAgentSpec` materializes the auxiliary capability specification from Definition 4.5, bounded by Rules A1–A4.
- `CoordinatorState` is the coordination agent’s session state containing the registry snapshot, interpreted request, candidate plan, feasibility evidence, and trace.
- The recovery records make durable state explicit: `SessionState`, `TaskState`, `PlanGeneration`, and `TaskCommitment` describe coordination progress, while `TaskAttemptRecord`, `LeaseRecord`, and `TerminalResultRecord` support invariant checks.
- `TraceEvent` records auditable observations across interpretation, authorization, delegation, and artifact handling.

Each of the five properties stated in Chapter 4 (justified acceptance, simplicity, bounded auxiliary synthesis, traceability, and explicit refusal) corresponds to a concrete decision or trace event owned by the `CoordinationAgent`, supported by communication evidence supplied through the SDK.

5.2 System Components

The proposed implementation is organized around a coordinating agent above a middleware SDK. The `CoordinationAgent` is the planner and authorizer. It receives user requests, maintains coordination state, requests linguistic interpretation when needed, constructs candidate plans, applies deterministic feasibility checks, decides whether to authorize or refuse the plan, dispatches authorized work through the SDK, and aggregates the final result.

`CoordinationSdk` is the communication and runtime substrate. It is the facade through which agents are registered and invoked. The SDK may use private helper classes internally, but adapter inheritance and transport details are not part of the public API. The SDK can reject invalid configuration, malformed registrations, missing handlers, invalid Agent Cards, and transport-level failures, but it does not decide whether a user request is feasible as a multi-agent plan.

The private SDK subsystems keep communication responsibilities simple. The Registry and Capability Index maintain admitted agent records and searchable capability metadata. The A2A Gateway fetches and validates Agent Cards, sends A2A tasks, and normalizes remote status, artifacts, cancellations, timeouts, and failures. The Local Agent Runtime wraps Python functions, scripts, classes, symbolic planners, and deterministic tools. The Lingo Agent Runtime wraps Lingo apps, flows, tools, state machines, and guardrails when linguistic behavior is required. The SDK Trace Store records communication and runtime observations. The Artifact Normalizer converts outputs into validator-friendly records.

5.3 Agent Registration and Communication Boundary

The SDK admits three kinds of agents through one communication boundary. A remote A2A agent is registered from an Agent Card URL. `AgentAdmissionPolicy` checks allowed origin, HTTPS policy, locally assigned trust, required credential schemes, and the explicit development-only HTTP exception. Effective trust is never accepted from the remote card. A credential provider supplies ephemeral request headers, while reports retain only scheme names. The SDK preserves card security schemes, canonical skill identifiers, endpoints, and modes in the internal registry entry.

A local agent is registered with `register_local_agent(name, capabilities, handler, ...)`. The handler may be a Python function, script wrapper, class, symbolic planner, or deterministic tool. The local agent does not need to know about A2A. The SDK supplies the same identity, capability, modality, authority, invocation, and validation metadata that a remote Agent Card would otherwise provide. During dispatch, the SDK invokes the local handler through the Local Agent Runtime and records the result as a normal task observation.

Table 5.1: Implementation components mapped to theoretical concepts.

Component	Role in the implementation
CoordinationAgent	Owens interpretation workflow, direct matching, decomposition, feasibility checks, auxiliary decisions, authorization, and final result construction.
CoordinationSdk	Exposes middleware operations for registration, registry inspection, capability indexing, task dispatch, artifact normalization, runtime failure reporting, traces, and session reset.
FeasibilityAnalyzer	Applies deterministic checks for the CoordinationAgent; it is not part of the SDK's communication boundary.
A2A Gateway	Handles Agent Card ingestion and A2A-compatible communication with remote agents.
Local Agent Runtime	Wraps Python handlers and deterministic local tools as SDK-managed agent capabilities.
Lingo Agent Runtime	Wraps optional Lingo apps, flows, tools, state machines, and guardrails as ordinary SDK capabilities.
SDK Trace Store	Tracks communication events, runtime failures, task status, and artifact observations.
Artifact Normalizer	Converts remote, local, and linguistic outputs into records usable by validators and aggregators.
Auxiliary Capability Factory	Used by the CoordinationAgent to specify temporary lightweight linguistic agents when justified.
Coordination Store	Persists session events, leases, fencing tokens, plan generations, task commitments, attempts, artifacts, terminal results, and recovery state.

A linguistic agent is registered through the SDK's linguistic-agent registration method. The wrapped object may be a Lingo app, flow, structured generation routine, state machine, tool set, or guardrail-bearing conversational process. The SDK exposes the linguistic agent as an ordinary capability provider. Lingo remains optional and internal to that capability. The `CoordinationAgent` may invoke the linguistic capability through `sdk.send_task(...)` when it needs interpretation, extraction, classification, explanation, or candidate planning.

No registered agent communicates directly with another registered agent through application-level shortcuts. The `CoordinationAgent` also does not bypass the SDK to reach remote transports, local handlers, or Lingo runtimes. This preserves one communication boundary while keeping planning and feasibility outside the SDK.

5.4 Minimum Data Structures

The implementation can be described through two public surfaces and a set of typed conceptual records. The records are represented as Pydantic models in the prototype so that Lingo structured generation, deterministic validation, SDK registration, coordination-agent planning, and test fixtures use the same schema surface. The records are not tied to a deployment platform; they define the information that the prototype must preserve.

`CoordinationSdk:`

```
register_a2a_agent(agent_card_url, trust_level="standard")
register_local_agent(name, capabilities, handler, ...)
register_linguistic_agent(name, capabilities, lingo_app_or_flow, ...)
registry_snapshot()
capability_index()
send_task(agent_id, task)
trace()
reset_session()
```

`CoordinationAgent:`

```
coordinate(user_request, context=None)
check_feasibility(problem_or_plan)
trace()
```

`AgentRegistryEntry:`

```
agent_id
name
agent_kind
service_endpoint
invocation_endpoint
```

- description
- skills
- input_modes
- output_modes
- status
- trust_level
- validation_contract
- source_card

CapabilityRequirement:

- name
- requirement_id
- capability_id
- description
- side_effect_class
- input_schema
- output_schema
- input_modes
- output_modes
- constraints
- auxiliary_eligible
- required_trust_level
- validation_contract

ConstraintSpec:

- constraint_id
- source
- path
- operator
- expected
- requirement_id
- task_id

ValidationContract:

- json_schema
- required_artifacts
- artifact_kinds
- required_fields
- evidence_types
- validator_id

CompletionContract:
 required_task_states
 required_artifacts
 require_all_task_validators

ProblemRequest:
 user_goal
 requirements
 constraints
 required_artifacts
 context

TaskSpec:
 task_id
 requirement_name
 requirement_id
 capability_id
 assigned_to
 auxiliary_spec_id
 depends_on
 expected_artifacts
 validation_contract

FeasibilityReport:
 feasible
 matched_agents
 generated_nlp_agents
 missing_capabilities
 risks
 explanation
 evidence

SolutionProposal:
 tasks
 generated_nlp_agents
 selected_agents
 execution_order
 expected_artifacts
 completion_criteria
 completion_contract
 constraint_evidence

GeneratedNlpAgentSpec:

- purpose
- input_schema
- output_schema
- method
- validation_rule
- lifecycle
- authority_bounds
- persists

CoordinatorState:

- registry_snapshot
- interpreted_request
- candidate_plan
- feasibility_evidence
- trace

TraceEvent:

- event_type
- message
- session_id
- plan_id
- task_id
- attempt_id
- source
- data
- timestamp

SessionState:

- new | planning | authorized | dispatching
- recovering | completed | failed | infeasible

TaskState:

- pending | running | completed | failed
- timeout | skipped | unknown

LeaseRecord:

- session_id
- holder_id
- fencing_token

```
expires_at
heartbeat_at
```

TaskAttemptRecord:

```
session_id
plan_id
task_id
attempt_id
idempotency_key
coordinator_id
fencing_token
state
```

TaskExecutionResult:

```
task_id
agent_id
agent_kind
status
output
artifacts
error
metadata
```

CoordinationRunResult:

```
status
plan_result
task_results
artifacts
explanation
trace
```

The CoordinationSdk public surface is intentionally limited to registration, inspection, dispatch, trace access, and session control. It hides the differences among remote A2A agents, local Python agents, and Lingo-backed linguistic agents while preserving enough metadata for the coordination agent to apply feasibility rules uniformly. The CoordinationAgent public surface contains the complete coordination workflow and feasibility query. The AgentRegistryEntry materializes the discoverable agent set. The CapabilityRequirement and ProblemRequest represent the user's goal, constraints, required capabilities, and required artifacts. The SolutionProposal is a candidate plan, not an executable plan until accepted. The FeasibilityReport explains whether the coordination agent can solve the problem and why. TaskExecutionResult and

`CoordinationRunResult` separate runtime observations from the pre-execution feasibility decision. The `GeneratedNlpAgentSpec` represents a temporary linguistic capability, such as a classifier or extractor, that can be created for the current solution without becoming a permanent system agent. The `CoordinatorState` and `TraceEvent` records provide the auditable ledger required by the traceability property.

5.5 Coordinator Workflow

The application-level coordinator is `CoordinationAgent`. It talks to the distributed system only through `CoordinationSdk`, but it keeps planning and authorization outside the SDK. This means the SDK is reusable for non-coordinating clients, while the thesis contribution remains concentrated in the coordination agent's feasibility-aware workflow.

```
sdk = CoordinationSdk(...)
analyzer = FeasibilityAnalyzer(...)
agent = CoordinationAgent(
    sdk=sdk,
    feasibility_analyzer=analyzer,
    linguistic_interpreter="optional-sdk-agent-id",
    auxiliary_factory=optional_factory,
    artifact_aggregator=aggregator,
)

sdk.register_a2a_agent(agent_card_url, trust_level="standard")
sdk.register_local_agent(name, capabilities, handler, ...)
sdk.register_linguistic_agent(name, capabilities, lingo_app_or_flow, ...)

result = agent.coordinate(user_request, context=session_context)
```

Inside `coordinate`, the `CoordinationAgent` performs the simple-first decision process. It asks the SDK for the current registry and capability index, transforms or admits the user's message into a typed `ProblemRequest`, attempts direct capability matching, requests candidate decomposition only when direct delegation is insufficient, checks the proposal with the `FeasibilityAnalyzer`, adds a bounded auxiliary specification only when the feasibility model permits it, and dispatches authorized tasks through the SDK.

```
CoordinationAgent.coordinate(problem):
    registry = sdk.registry_snapshot()
    capabilities = sdk.capability_index()
    request = interpret_or_admit_problem(problem, sdk)
```

```

candidate_plan = direct_or_decomposed_plan(request, capabilities, sdk)
report = feasibility_analyzer.check(request, registry, candidate_plan)

if report.has_auxiliary_eligible_gap:
    auxiliary = auxiliary_factory.specify(report.gap)
    candidate_plan = candidate_plan.with_auxiliary_agent(auxiliary)
    report = feasibility_analyzer.check(request, registry, candidate_plan)

if report.is_not_feasible:
    return structured_infeasibility(report)

results = [
    sdk.send_task(task.assigned_to, task)
    for task in candidate_plan.execution_order
]
return artifact_aggregator.combine(results)

```

Task dispatch remains polymorphic inside the SDK but not visible to the coordination agent. If the selected executor is a remote A2A agent, the A2A Gateway sends the task and normalizes the remote response. If it is a local agent, the Local Agent Runtime invokes the registered handler and converts the result into the same task observation shape. If it is a Lingo-backed agent, the Lingo Agent Runtime executes the approved app, flow, tool, or state machine and returns a normalized artifact. In all cases, planning infeasibility is structured data returned by the CoordinationAgent. SDK exceptions are reserved for invalid SDK configuration or genuine transport/runtime failures.

This workflow keeps the LLM useful but bounded. Lingo may supply typed context handling and structured proposal mechanisms through SDK-registered linguistic capabilities, but it does not alone decide feasibility. Deterministic checks remain responsible for explicit properties such as availability, modality compatibility, declared skills, authority, and missing capabilities. Lingo’s native tool-calling interface may be useful for controlled experiments that intentionally remove symbolic checks, but the main hybrid implementation uses developer-controlled structured output so that symbolic authorization remains explicit.

5.6 Local and Linguistic Agent Specializations

The SDK treats local and linguistic agents as specializations of the same communication boundary rather than as separate architectural paths. A local agent is appropriate when the capability is deterministic, symbolic, scriptable, or already implemented in Python. Examples include a rule-based planner, a schema validator, a database wrapper, a file transformer, or a

mathematical tool. The SDK supplies the communication wrapper, so the local agent author focuses on the capability contract and handler behavior.

A linguistic agent is appropriate when the capability transforms, interprets, classifies, summarizes, or produces language-mediated artifacts. It may be implemented with Lingo, an LLM, a smaller NLP model, rules, retrieval, or a hybrid pipeline. In the prototype, Lingo can realize both broad and narrow linguistic work: `Engine.create` can extract a typed object, `Engine.decide` can perform a bounded binary classification, `Engine.choose` can select from an enumerated set, `Flow` can encode a procedural workflow, and a `StateMachine` can constrain a multi-step dialogue.

Generated NLP agents are solution-local. The `CoordinationAgent` includes their specifications in the `SolutionProposal`, including purpose, method, expected input, expected output, validation rule, and lifecycle. They do not automatically enter the registry. Promotion to a persistent registered agent would require a separate SDK admission process.

5.7 Architecture Diagram

Figure 5.1 summarizes the implementation structure. The diagram is intentionally abstract enough to support later coding choices while still showing that coordination logic belongs to `CoordinationAgent`, while communication and runtime adaptation belong to `CoordinationSdk`.

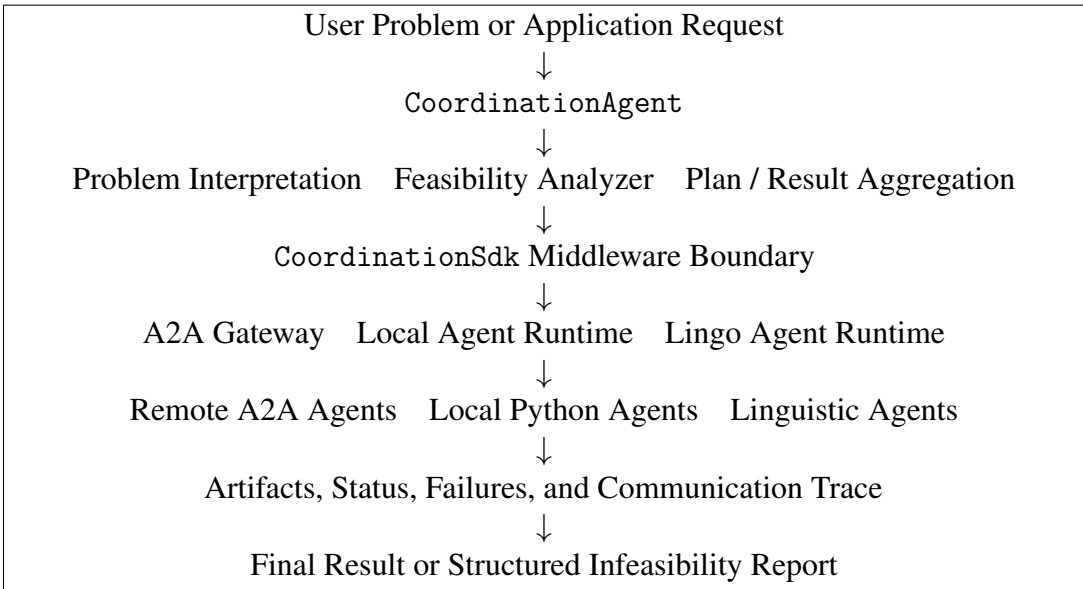


Figure 5.1: `CoordinationAgent` over SDK middleware for A2A-compatible coordination across remote, local, and linguistic agents.

5.8 Execution Sequence

The execution sequence begins when the user or application submits a problem request to `CoordinationAgent.coordinate`. The coordination agent asks the SDK for the current registry snapshot and capability index. It obtains a typed problem representation from a structured caller or by invoking a registered linguistic capability through the SDK. The coordination agent then attempts direct matching before considering decomposition. The `FeasibilityAnalyzer` checks the candidate plan and either approves it, permits an auxiliary NLP specification to be added, or rejects it as infeasible. Approved tasks are dispatched through `sdk.send_task(...)`. The SDK chooses the correct internal runtime, records status and failures, normalizes artifacts, and exposes communication trace events. The coordination agent aggregates the authorized outputs or returns an infeasibility report.

1. User or application submits a problem and constraints to `CoordinationAgent`.
2. Coordination agent reads registry and capabilities through `CoordinationSdk`.
3. Coordination agent obtains or constructs a typed `ProblemRequest`.
4. Coordination agent attempts direct capability matching before decomposition.
5. Coordination agent requests candidate planning only when direct matching is insufficient.
6. Feasibility Analyzer validates skills, modalities, dependencies, authority, and evidence contracts.
7. Auxiliary Capability Factory adds a temporary specification only when justified.
8. Coordination agent dispatches authorized tasks through `sdk.send_task(...)`.
9. SDK records runtime status, errors, artifacts, and communication evidence.
10. Coordination agent returns the final result or a structured infeasibility report.

Figure 5.2: Execution sequence for `CoordinationAgent`-mediated problem solving.

5.9 Failure and Infeasibility Handling

The implementation treats infeasibility as a first-class coordination-agent result. If no remote, local, linguistic, or approved auxiliary capability can satisfy a required subtask, the

`CoordinationAgent` returns a structured report. If an agent is unavailable, the coordination agent may select another compatible agent from the SDK registry; if no replacement exists, it reports the missing dependency. If modalities are incompatible, the report identifies which input or output mode cannot be bridged. If a linguistic planner proposes a decomposition that violates explicit constraints, the `FeasibilityAnalyzer` rejects it.

The SDK normalizes runtime failures rather than planning failures. A2A transport failures, local handler exceptions, Lingo execution failures, invalid artifacts, cancellations, and timeouts are recorded as structured observations linked to the relevant task and agent. This behavior is essential for simplicity and trust. The system should not create complex plans merely to appear capable. It should coordinate only when the available and generated capabilities justify the solution. Otherwise, the coordination agent should explain the boundary clearly. Lingo guardrails may detect obvious unsafe, irrelevant, or ambiguous inputs early, but such reflexes are treated as advisory preprocessing rather than as replacements for the feasibility verdict.

5.10 Lease-Backed Replicated Coordinator Execution

The implemented crash-fault-tolerance path preserves the same logical coordination authority while allowing several coordinator processes to serve different sessions and to recover a session after the previous owner stops making progress. The key design point is that replication applies to the coordinator process, not to the authorization role. Each session still has one current lease holder, one authorized plan state per generation, one task-attempt history, and at most one terminal result.

The `CoordinationStore` interface separates durable coordination state from the coordination algorithm. The JSONL adapter keeps local tests and simple demonstrations lightweight. The PostgreSQL adapter uses transactional tables for sessions, leases, append-only events, registry snapshots, plan generations, task commitments, task attempts, artifacts, and terminal results. Its schema enforces key invariants: a session has at most one terminal result, task results require a prior attempt record, stale fencing tokens are rejected, and idempotency keys can be resolved before retrying a dispatch.

Coordinator replicas identify themselves with `COORDINATION_COORDINATOR_ID`. Before planning, authorization, dispatch, recovery, or terminal writes, `coordinate` and `resume_session` acquire a session lease. The store assigns a monotonically increasing fencing token whenever a lease is acquired after expiry or release. Every fenced write carries the lease token, and the PostgreSQL backend rejects stale tokens after takeover. Service endpoints translate live lease conflicts into HTTP 409, making ownership contention visible to clients rather than silently reassigning the session.

Dispatch is idempotency-aware. Each attempt uses the key `session_id + plan_id +`

Table 5.2: Scope of implemented coordination evidence.

Level	Implemented evidence	Claim boundary
Local prototype	JSONL-backed sessions, deterministic feasibility checks, SDK dispatch, trace reconstruction, and resume from stored events.	Shows the language-symbolic boundary and recovery for simple local sessions.
Replicated coordinator	PostgreSQL store, leases, fencing tokens, idempotency keys, per-attempt provenance, stale-write rejection, HTTP 409 lease conflict responses, and Docker failover evidence.	Supports crash-fault-tolerant replicated coordinator execution for the measured cases.
Decentralized or adversarial coordination	Not implemented: no consensus protocol, adversarial registry defense, malicious Agent Card validation, or Byzantine coordinator tests.	Remains outside scope and is not claimed.

task_id + attempt_id; the key is persisted before SDK dispatch and is included in the task payload’s _coordination metadata together with the coordinator id, fencing token, and plan generation. The SDK caches completed local and linguistic results by idempotency key, and the deterministic A2A fixture agent returns a prior result when the same key is observed again. The CapabilityRequirement.side_effect_class field distinguishes read_only, idempotent, unsafe, and unknown work. Automatic redispach is limited to read-only or idempotent attempts. When takeover observes an expired in-flight task whose side effects are unknown or unsafe, the coordinator records an unknown task result and fails explicitly rather than blindly repeating external work.

Recovery reconstructs session state from the store. Completed tasks are skipped, pending tasks whose dependencies are satisfied can resume, and terminal results are returned without replanning. Replanning is deliberately conservative: a new plan generation must be recorded rather than silently mutating the old authorized plan, and it is only justified when the prior task lacks a terminal result, the original executor is unavailable, a compatible replacement exists in a fresh registry snapshot, and the feasibility analyzer accepts the new proposal. The implemented Docker evidence covers lease conflict, takeover after an abandoned attempt, process exit after external dispatch of unknown-side-effect work, stale fencing rejection, and PostgreSQL invariant checks; broader dynamic-registry replanning remains a future evaluation case.

This design keeps the theory’s centralized authorization semantics intact. The system may have several coordinator processes, but the session still has one logical authorization record and one current commitment state. Fault tolerance is therefore an implementation

property of state durability, lease ownership, retry discipline, idempotency, and failover. It is not a shift to decentralized consensus and does not address Byzantine coordinators or malicious registries.

5.11 Chapter Synthesis

The design specifies a distributed multi-agent environment with a clear split between coordination and communication. `CoordinationAgent` owns planning, feasibility, authorization, auxiliary decisions, and aggregation. `CoordinationSdk` owns A2A interoperability, local runtime invocation, Lingo runtime invocation, registry normalization, task dispatch, artifact normalization, communication failures, and trace access. Agents remain independently deployable and vendor independent. A2A provides the interoperability backbone through Agent Cards and task exchange. Local Python agents and Lingo-backed linguistic agents are wrapped by the SDK so that they participate through the same registry, dispatch, artifact, and trace mechanisms as remote A2A agents. The evaluation in Chapters 3 and 6 examines whether the implementation preserves both communication-boundary compliance and coordination-ownership compliance in practice.

Chapter 6

Evaluation and Results

6.1 Evidence Status

This chapter distinguishes corrected version 0.2 preflight evidence from final study evidence. Reports produced by the earlier implementation are legacy artifacts and are excluded from every table and conclusion below. The comparative 36-case local-model study is not yet scored because the independent label sign-off is incomplete. Consequently, this draft reports implementation verification but does not report model accuracy, baseline superiority, or an answer to RQ4 based on the planned comparative study.

Table 6.1: Corrected version 0.2 preflight gates.

Gate	Observed	Required
Python tests	78 passing	all passing
Overall source coverage	55%	at least 85%
Deterministic scenarios	7/7	7/7
Docker A2A checks	11/11	11/11
PostgreSQL distributed checks	5/5	5/5
Official A2A sample interoperability	1/1	passing
Ruff and mypy	passing	passing
Independent corpus sign-off	incomplete	complete
Local-model outputs	0/540	540/540

The coverage gate is the principal automated quality blocker. Aggregate coverage is not used to conceal critical gaps: the corrected measurement reports 88% for the coordination agent, 83% for feasibility, 77% for trace validation, 42% for the PostgreSQL store, and no pytest coverage in the newly added corpus and study runners. Thus the release is not defense-ready even though the executed tests pass.

6.2 Fail-Closed Symbolic Boundary

Regression tests show that a promised artifact does not make a validatorless task verifiable; legacy textual constraints become unresolved typed constraints and force refusal; missing JSON-pointer paths, false predicates, canonical capability-ID mismatch, incomplete mode coverage, and nested JSON Schema type violations are rejected. Effective trust is assigned by local admission policy rather than copied from a remote Agent Card. Cards requiring an unavailable credential scheme are refused, and plain HTTP requires an explicit development exception.

Candidate plans remain non-authoritative. Direct delegation is selected only if it is feasible; otherwise bounded candidates are checked and ranked deterministically by task count, auxiliary count, DAG depth, and stable task identifiers. These tests support prototype correctness of the implemented predicates, not mathematical proof of arbitrary plan correctness.

6.3 Dependency, Recovery, and Trace Behavior

Execution tests confirm that a task is dispatched only after every declared prerequisite completes. Failure, timeout, refusal, or unknown outcome skips descendants while independent branches continue. Consumers receive artifacts only from declared dependencies. On takeover, unsafe unknown work is recorded as unknown and is not blindly redispached. Completion additionally requires the typed completion contract.

Concurrent-session tests show that returned traces contain only persisted events for their session. Authorization precedes attempt start, SDK transport observations are persisted with session, plan, task, and attempt identity, and a trace validator checks ordering and terminal observations. Registration diagnostics remain outside operational session traces.

6.4 Corrected End-to-End Harnesses

The seven deterministic statuses match their references: three successful completions, one authorized runtime failure, and three pre-dispatch infeasibility decisions. The privacy case now derives email and phone values from transcript text; the answers are no longer supplied as separate fixture fields. The task-local auxiliary lifecycle expires after use.

The corrected Docker harness passes eleven checks covering health, registry discovery, direct and decomposed A2A execution, missing capability, insufficient authority, incompatible modes, invalid artifacts, timeout, registry outage, and session replay. The decomposed case originally exposed a stale assumption that every prior artifact should be forwarded. The

corrected planner declares dependencies from canonical mode flow, and the harness now verifies that the final consumer receives only its declared predecessor artifact.

The PostgreSQL harness passes five checks: live lease conflict, takeover after an abandoned read-only attempt, process exit after external dispatch of unknown-side-effect work, stale-fence rejection, and database invariant rejection. It records zero duplicate dispatches and repeated effects in these cases, one lease conflict, one stale-write rejection, terminal correctness, and measured recovery latency. These observations do not establish consensus or cover all crash windows.

An additional test launches the vendored official A2A Hello World sample as an independent server, ingests its Agent Card, preserves its `echo_bot` skill identifier, authorizes the matching task, and receives its remote artifact. This establishes one concrete interoperability path; it is not evidence of universal A2A compatibility.

6.5 Comparative Study Gate

The public corpus contains 36 cases and has hash `b9fd8b6f8398eaf33b8676e14288f0d3c4956cca5a9b915ca7b68990167c9c1d`. All three required models are installed and the LM Studio server is reachable. The sign-off record is deliberately marked unapproved. The collection runner therefore refuses to produce the 540 submitted outputs, and the scoring program independently refuses to read hidden labels. This is a successful blinding control, not an empirical result.

RQ1–RQ3 receive implementation-level support from the corrected boundary and harnesses. RQ4 remains open until sign-off, collection, scoring, uncertainty analysis, and a clean evidence freeze are complete. The abstract and conclusion therefore contain no numerical comparative claim.

Chapter 7

Discussion

7.1 Interpretation

The contribution is an integrative authorization/refusal boundary. Established mechanisms—typed schemas, planning constraints, access policy, task DAGs, leases, fencing, idempotency, and protocol adapters—are combined so that linguistic output cannot silently acquire executable authority. The new contribution claimed here is their operational integration and traceable refinement into one prototype, not invention of each mechanism or formal verification of arbitrary agents.

The LLM is useful where wording varies and decomposition is not known in advance. It is not an authority. Stable capability identifiers, local trust assignment, typed constraints, validation contracts, and completion contracts determine what may be dispatched. A structured rule-only oracle remains a useful upper bound because it shows what can be achieved when interpretation is already supplied; it is not a fair replacement for an end-user system receiving natural language.

7.2 Guarantees and Non-Guarantees

Within the tested implementation, authorization precedes dispatch, unresolved constraints fail closed, declared failed dependencies block descendants, independent branches may continue, session traces are isolated, and stale fenced writes are rejected. These are implementation properties supported by tests and measured traces. They are not mathematical proofs over every deployment, scheduler, transport, or database failure.

Contract validity is weaker than semantic truth. JSON Schema can establish nested type and required-field conformance; a validator can establish a domain-specific property only if that property is encoded. Neither implies that an open-ended summary is complete or that a physical action is safe. The trusted registry and honest-agent assumptions remain explicit.

The admission boundary controls allowed origins, HTTPS policy, locally assigned trust, required credential schemes, and request-header propagation. It does not provide cryptographic identity, attestation, or defense against a malicious admitted agent. Plain HTTP is enabled only for explicit development fixtures.

Lease and fencing recovery is not consensus. PostgreSQL supplies one serialization point; replicas coordinate through the shared store. The evidence covers selected crash-fault windows and unknown-side-effect refusal, not partitions with competing databases, Byzantine replicas, or arbitrary distributed failures.

7.3 Threats to Validity

Construct validity is protected by separating feasibility, execution, artifact, trace, and recovery measures. Internal validity remains threatened by model nondeterminism, prompt sensitivity, baseline implementation choices, and fixture ownership. The planned shared raw proposal and paired cases reduce, but cannot remove, those threats. External validity is limited to 36 designed cases, three small local models, one official sample, controlled fixture agents, and measured crash faults. Conclusion validity requires five seeds, model-stratified reporting, exact paired tests, effect sizes, and bootstrap intervals; until those data exist, comparative conclusions are withheld.

7.4 Defense Readiness

The corrected implementation is materially stronger than the legacy prototype, but this draft is not ready to freeze. Coverage thresholds, independent label approval, 540 model outputs, complete scoring, final institutional metadata, PDF visual inspection, a clean source commit, and Docker digests remain open. Treating those open gates as results would violate the thesis's own authorization principle.

Chapter 8

Conclusions and Future Work

The thesis proposes a protocol-adaptable boundary between linguistic proposal and executable multi-agent coordination. RQ1 is answered at the design and prototype level: canonical capabilities, typed constraints, task dependencies, authority, validation and completion contracts, artifacts, attempts, and persisted trace identity must be explicit to justify dispatch. RQ2 is answered architecturally: language interprets and proposes, while deterministic components resolve identity, evaluate enforceable predicates, authorize, refuse, and record. RQ3 is answered through bounded task-local auxiliary specifications whose method, schema, validator, read-only authority, and lifecycle are explicit.

RQ4 is not yet answered by the corrected comparative study. The 36-case corpus, three-model/five-seed collector, configuration scorer, paired tests, and provenance records are implemented, but independent label approval has not been supplied and collection is correctly blocked. No legacy numerical result is substituted. The current evidence supports prototype behavior in unit, deterministic, Docker A2A, official-sample, and PostgreSQL crash-fault checks; it does not support broad empirical superiority.

The principal scientific claim is therefore deliberately narrow. The implementation demonstrates how established symbolic, security, persistence, and interoperability mechanisms can be integrated so that unsupported linguistic proposals are refused before action and process evidence survives recovery. It does not prove semantic truth, production safety, malicious-agent resistance, decentralized consensus, Byzantine tolerance, or universal protocol interoperability.

Before submission, future work is operational rather than speculative: raise critical-module coverage to the declared gates; obtain independent corpus sign-off; collect and preserve all 540 outputs; execute the paired analysis and ablations; regenerate every thesis table from frozen reports; supply tutor, tribunal, date, commit, and repository metadata; compile and visually inspect the PDF; and freeze source, corpus, evidence, and thesis at one clean tagged commit. Longer-term research should add cryptographic admission, adversarial registries, independently developed domain agents, semantic validators, alternate-agent

replanning, and broader deployment populations.

Appendix A

Reproduction and Artifact Manifest

The repository root contains REPRODUCING.md, evidence-manifest.json, the locked dependency graph, Docker configurations, and the versioned corpus. Legacy reports are explicitly excluded. A final accepted manifest must contain the clean Git commit, Docker digests, corpus hash, run identifiers, and hashes of generated thesis tables.

A.1 Command Sequence

```
uv sync --locked --group dev
uv run ruff check src tests scripts
uv run mypy src/unified_multi_agent_coordination
uv run pytest --cov
uv run unified-coordination-scenarios --no-output
docker compose -f docker-compose.system.yml up --build \
    --abort-on-container-exit --exit-code-from system-tests
docker compose -f docker-compose.distributed.yml up --build \
    --abort-on-container-exit --exit-code-from distributed-system-tests
uv run unified-defense-study --check
```

Appendix B

Predicate Refinement and Claim–Evidence Matrix

Claim or predicate	Implementation refinement	Evidence required
Coverage	Canonical capability ID, full modes and schema coverage	Feasibility and ID-perturbation tests
Authority	Local trust order and admission policy	Remote-card and credential tests
Constraints	Typed source, JSON pointer, operator and expected value	Missing, type, false and unresolved cases
Verifiability	Task, requirement or capability validation contract	Validatorless and nested-schema tests
Dependency safety	Completed prerequisites and declared artifacts only	Failure/timeout/unknown branch tests
Traceability	Persisted session, plan, task and attempt events	Ordering, restart and isolation tests
Recovery	Lease, fencing, idempotency and unknown-side-effect policy	PostgreSQL crash-fault harness
RQ4 comparison	Blinded 36-case paired model study	Signed labels, 540 outputs and frozen analysis

Appendix C

Tribunal Questions and Defensible Boundaries

What is novel? The protocol-adaptable integration and operational refinement of a linguistic proposal boundary, symbolic authorization/refusal, durable evidence, admission policy, and measured recovery. Individual mechanisms are prior art.

Why use an LLM? Natural-language interpretation and candidate decomposition vary semantically; a structured rule oracle assumes that work is already done. The LLM remains non-authoritative.

What is guaranteed? The tested implementation enforces declared predicates and ordering under its trusted-code and trusted-store assumptions. It does not guarantee semantic truth or runtime success.

What is trusted? The coordinator code, durable store, locally configured trust policy, admitted registry descriptions, and contract validators. Malicious admitted agents remain outside scope.

Why is fencing not consensus? All replicas serialize through one PostgreSQL authority; there is no replicated log or quorum protocol.

How broad is A2A interoperability? Controlled fixtures plus one vendored official sample. This is evidence of concrete compatibility, not universal conformance.

Why are comparative conclusions absent? Independent corpus approval and the complete repeated study are release gates. Reporting legacy numbers would invalidate the corrected evaluation.

Bibliography

- [1] A2A Project. Agent2Agent Protocol. <https://github.com/a2aproject/A2A>, 2026. Accessed 2026-07-05.
- [2] Agent Communication Protocol Project. Agent Communication Protocol Documentation. <https://agentcommunicationprotocol.dev/introduction/welcome>, 2026. Accessed 2026-07-05.
- [3] Agent Network Protocol Project. Agent Network Protocol. <https://agent-network-protocol.com/>, 2026. Accessed 2026-07-05.
- [4] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Accessed 2026-07-05.
- [5] J. L. Austin. *How to Do Things with Words*. Oxford University Press, 1962.
- [6] Franz Baader, Diego Calvanese, Deborah L. McGuinness, Daniele Nardi, and Peter F. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation, and Applications*. Cambridge University Press, 2003.
- [7] Fabio Luigi Bellifemine, Giovanni Caire, and Dominic Greenwood. *Developing Multi-Agent Systems with JADE*. Wiley, 2007.
- [8] Tarek R. Besold, Artur S. d’Avila Garcez, Sebastian Bader, Howard Bowman, Pedro Domingos, Pascal Hitzler, et al. Neural-symbolic learning and reasoning: A survey and interpretation. arXiv:1711.03902, 2017.
- [9] Olivier Boissier, Rafael H. Bordini, Jomi F. Hubner, Alessandro Ricci, and Andrea Santi. Multi-agent oriented programming with jacamo. *Science of Computer Programming*, 78(6):747–761, 2013.
- [10] Rafael H. Bordini, Michael Fisher, Willem Visser, and Michael Wooldridge. Verifying multi-agent programs by model checking. *Autonomous Agents and Multi-Agent Systems*, 12:239–256, 2006.

- [11] Rafael H. Bordini, Jomi F. Hubner, and Michael Wooldridge. *Programming Multi-Agent Systems in AgentSpeak Using Jason*. Wiley, 2007.
- [12] Michael E. Bratman. *Intention, Plans, and Practical Reason*. Harvard University Press, 1987.
- [13] Rodney A. Brooks. A robust layered control system for a mobile robot. *IEEE Journal on Robotics and Automation*, 2(1):14–23, 1986.
- [14] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- [15] G. Chang, E. Lin, C. Yuan, R. Cai, B. Chen, X. Xie, and Y. Zhang. Agent network protocol technical white paper. arXiv:2508.00007, 2025.
- [16] Philip R. Cohen and Hector J. Levesque. Intention is choice with commitment. *Artificial Intelligence*, 42(2–3):213–261, 1990.
- [17] Michele Colledanchise and Petter Ogren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.
- [18] Daniel D. Corkill. Blackboard systems. *AI Expert*, 6(9):40–47, 1991.
- [19] CrewAI. CrewAI Documentation: Introduction. <https://docs.crewai.com/en/introduction>, 2026. Version 1.14.1, accessed 2026-07-05.
- [20] Artur S. d’Avila Garcez, Marco Gori, Luis C. Lamb, Luciano Serafini, Michael Spranger, and Son N. Tran. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. *Journal of Applied Logics*, 6(4):611–632, 2019.
- [21] Louise A. Dennis, Michael Fisher, Marija Slavkovik, and Matt Webster. Formal verification of ethical choices in autonomous systems. *Robotics and Autonomous Systems*, 77:1–14, 2016.
- [22] Virginia Dignum. *A Model for Organizational Interaction: Based on Agents, Founded in Logic*. PhD thesis, Utrecht University, 2004.
- [23] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar. A survey of agent interoperability protocols: Model context protocol, agent communication protocol, agent-to-agent protocol, and agent network protocol. arXiv:2505.02279, 2025.

- [24] Kutluhan Erol, James Hendler, and Dana S. Nau. Htn planning: Complexity and expressivity. In *Proceedings of the Twelfth National Conference on Artificial Intelligence*, pages 1123–1128, 1994.
- [25] Jacques Ferber. *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*. Addison-Wesley, 1999.
- [26] Richard E. Fikes and Nils J. Nilsson. Strips: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.
- [27] Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. Kqml as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management*, pages 456–463, 1994.
- [28] Ferdinando Fioretto, Enrico Pontelli, and William Yeoh. Distributed constraint optimization problems and applications: A survey. *Journal of Artificial Intelligence Research*, 61:623–698, 2018.
- [29] Michael Fisher, Louise A. Dennis, and Matt Webster. Verifying autonomous systems. *Communications of the ACM*, 56(9):84–93, 2013.
- [30] Foundation for Intelligent Physical Agents. Fipa communicative act library specification. <http://www.fipa.org/specs/fipa00037/>, 2002.
- [31] Maria Fox and Derek Long. Pddl2.1: An extension to pddl for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, 2003.
- [32] Stan Franklin and Art Graesser. Is it an agent, or just a program? a taxonomy for autonomous agents. In *Intelligent Agents III: Agent Theories, Architectures, and Languages*. Springer, 1997.
- [33] David Gelernter. Generative communication in linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, 1985.
- [34] Michael Gelfond and Vladimir Lifschitz. Classical negation in logic programs and disjunctive databases. *New Generation Computing*, 9:365–385, 1991.
- [35] Michael Georgeff, Barney Pell, Martha Pollack, Milind Tambe, and Michael Wooldridge. The belief-desire-intention model of agency. In *Intelligent Agents V: Agent Theories, Architectures, and Languages*. Springer, 1999.
- [36] Ilche Georgievski and Marco Aiello. Htn planning: Overview, comparison, and beyond. *Artificial Intelligence*, 222:124–156, 2015.

- [37] Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning: Theory and Practice*. Morgan Kaufmann, 2004.
- [38] Google Developers Blog. Announcing the agent2agent protocol. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>, 2025. Accessed 2026-07-05.
- [39] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. arXiv:2302.12173, 2023.
- [40] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.
- [41] Grupo de Inteligencia Artificial, Universidad de La Habana. Lingo: A python library for context engineering. <https://github.com/gia-uh/lingo>; PyPI package <https://pypi.org/project/lingo-ai/>, 2026. Version 2.0.2, accessed 2026-06-23.
- [42] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichang Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. arXiv:2402.01680, 2024.
- [43] Koen V. Hindriks. Programming rational agents in goal. In *Multi-Agent Programming: Languages, Tools and Applications*. Springer, 2009.
- [44] Pascal Hitzler, Md Kamruzzaman Sarker, and Aaron Eberhart, editors. *Neuro-Symbolic Artificial Intelligence: The State of the Art*. IOS Press, 2022.
- [45] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, et al. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.
- [46] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, et al. Metagpt: Meta programming for a multi-agent collaborative framework. arXiv:2308.00352, 2023.
- [47] X. Hou, Y. Zhao, S. Wang, and H. Wang. Model context protocol: Landscape, security threats, and future research directions. arXiv:2503.23278, 2025.
- [48] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. arXiv:2201.07207, 2022.

- [49] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, et al. Inner monologue: Embodied reasoning through planning with language models. arXiv:2207.05608, 2022.
- [50] IBM Research. Agent communication protocol. <https://research.ibm.com/projects/agent-communication-protocol>, 2025. Accessed 2026-06-17.
- [51] Nicholas R. Jennings. Coordination techniques for distributed artificial intelligence. In *Foundations of Distributed Artificial Intelligence*. Wiley, 1996.
- [52] Nicholas R. Jennings. On agent-based software engineering. *Artificial Intelligence*, 117(2):277–296, 2000.
- [53] Nicholas R. Jennings, Peyman Faratin, Alessio R. Lomuscio, Simon Parsons, Michael Wooldridge, and Carles Sierra. Automated negotiation: Prospects, methods and challenges. *Group Decision and Negotiation*, 10:199–215, 2001.
- [54] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.
- [55] E. Karpas, O. Abend, Y. Belinkov, B. Lenz, O. Lieber, N. Ratner, et al. Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. arXiv:2205.00445, 2022.
- [56] Yannis Labrou and Tim Finin. A proposal for a new kqml specification. Technical report, University of Maryland Baltimore County, 1997.
- [57] LangChain. LangGraph Overview. <https://docs.langchain.com/oss/python/langgraph/overview>, 2026. Accessed 2026-07-05.
- [58] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [59] LF AI and Data Foundation. Acp joins forces with a2a under the linux foundation’s lf ai and data. LF AI and Data community blog, 2025. Accessed 2026-07-05.
- [60] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for mind exploration of large language model society. arXiv:2303.17760, 2023.

- [61] Zelong Li, Wenyue Hua, Hao Wang, He Zhu, and Yongfeng Zhang. Formal-llm: Integrating formal language and natural language for controllable llm-based agents. *arXiv preprint arXiv:2402.00798*, 2024.
- [62] Vladimir Lifschitz. What is answer set programming? *Proceedings of the AAAI Conference on Artificial Intelligence*, 22(1):1594–1597, 2008.
- [63] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. Llm+p: Empowering large language models with optimal planning proficiency. *arXiv:2304.11477*, 2023.
- [64] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, et al. Agent-bench: Evaluating llms as agents. *arXiv:2308.03688*, 2023.
- [65] Thomas W. Malone and Kevin Crowston. The interdisciplinary study of coordination. *ACM Computing Surveys*, 26(1):87–119, 1994.
- [66] John McCarthy and Patrick J. Hayes. Some philosophical problems from the standpoint of artificial intelligence. In *Machine Intelligence 4*. Edinburgh University Press, 1969.
- [67] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. Pddl: The planning domain definition language. Technical report, Yale Center for Computational Vision and Control, 1998.
- [68] Gregoire Mialon, Roberto Dessi, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, et al. Augmented language models: A survey. *Transactions on Machine Learning Research*, 2023.
- [69] Gregoire Mialon, Clementine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: A benchmark for general ai assistants. *arXiv:2311.12983*, 2023.
- [70] Microsoft. Introduction to semantic kernel. <https://learn.microsoft.com/en-us/semantic-kernel/overview/>, 2026. Accessed 2026-07-05.
- [71] Model Context Protocol. Specification 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25>, 2025. Accessed 2026-07-05.
- [72] Model Context Protocol. Draft Specification. <https://modelcontextprotocol.io/specification/draft>, 2026. Accessed 2026-07-05.

- [73] Pragnesh J. Modi, Wei-Min Shen, Milind Tambe, and Makoto Yokoo. Adopt: Asynchronous distributed constraint optimization with quality guarantees. *Artificial Intelligence*, 161(1–2):149–180, 2005.
- [74] Dana Nau, Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman. Shop2: An htn planning system. *Journal of Artificial Intelligence Research*, 20:379–404, 2003.
- [75] Allen Newell and Herbert A. Simon. Computer science as empirical inquiry: Symbols and search. *Communications of the ACM*, 19(3):113–126, 1976.
- [76] H. Penny Nii. Blackboard systems: The blackboard model of problem solving and the evolution of blackboard architectures. *AI Magazine*, 7(2):38–53, 1986.
- [77] Jeff Orkin. Three states and a plan: The ai of f.e.a.r. In *Game Developers Conference*, 2006.
- [78] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.
- [79] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [80] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, et al. Communicative agents for software development. arXiv:2307.07924, 2023.
- [81] Iyad Rahwan and Guillermo R. Simari, editors. *Argumentation in Artificial Intelligence*. Springer, 2009.
- [82] Anand S. Rao and Michael P. Georgeff. Modeling rational agents within a bdi-architecture. In *Proceedings of the Second International Conference on Principles of Knowledge Representation and Reasoning*, pages 473–484, 1991.
- [83] Anand S. Rao and Michael P. Georgeff. Bdi agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems*, pages 312–319, 1995.
- [84] Raymond Reiter. *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems*. MIT Press, 2001.

- [85] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.
- [86] Tuomas W. Sandholm. An implementation of the contract net protocol based on marginal cost calculations. In *Proceedings of the Eleventh National Conference on Artificial Intelligence*, pages 256–262, 1993.
- [87] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [88] John R. Searle. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press, 1969.
- [89] Onn Shehory and Sarit Kraus. Methods for task allocation via agent coalition formation. *Artificial Intelligence*, 101(1–2):165–200, 1998.
- [90] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. arXiv:2303.11366, 2023.
- [91] Yoav Shoham. Agent-oriented programming. *Artificial Intelligence*, 60(1):51–92, 1993.
- [92] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Aleworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021.
- [93] Munindar P. Singh. An ontology for commitments in multiagent systems. *Artificial Intelligence and Law*, 7:97–113, 1999.
- [94] Reid G. Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980.
- [95] The Linux Foundation. Linux foundation launches the agent2agent protocol project to enable secure, intelligent communication between ai agents. Linux Foundation press release, 2025. Accessed 2026-07-05.
- [96] Guy Theraulaz and Eric Bonabeau. A brief history of stigmergy. *Artificial Life*, 5(2):97–116, 1999.
- [97] K. T. Tran et al. Multi-agent collaboration mechanisms: A survey of large language models. arXiv:2501.06322, 2025.

- [98] Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models: A critical investigation. *arXiv:2305.15771*, 2023.
- [99] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [100] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv:2305.16291*, 2023.
- [101] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18:186345, 2024.
- [102] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [103] Gerhard Weiss, editor. *Multiagent Systems*. MIT Press, 2 edition, 2013.
- [104] Michael P. Wellman. A market-oriented programming environment and its application to distributed multicommodity flow problems. *Journal of Artificial Intelligence Research*, 1:1–23, 1993.
- [105] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2 edition, 2009.
- [106] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.
- [107] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv:2308.08155*, 2023.
- [108] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv:2404.07972*, 2024.
- [109] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv:2210.03629*, 2023.

- [110] Makoto Yokoo, Edmund H. Durfee, Toru Ishida, and Kazuhiro Kuwabara. The distributed constraint satisfaction problem: Formalization and algorithms. *IEEE Transactions on Knowledge and Data Engineering*, 10(5):673–685, 1998.
- [111] Xing Zhang, Yanwei Cui, Guanghui Wang, Qucy Wei Qiu, Ziyuan Li, Fangwei Han, Yajing Huang, Hengzhi Qiu, Bin Zhu, and Peiyang He. Verified multi-agent orchestration: A plan-execute-verify-replan framework for complex query resolution. *arXiv preprint arXiv:2603.11445*, 2026.
- [112] Yedi Zhang, Yufan Cai, Xinyue Zuo, Xiaokun Luan, Kailong Wang, Zhe Hou, Yifan Zhang, Zhiyuan Wei, Meng Sun, Jun Sun, Jing Sun, and Jin Song Dong. Position: Trustworthy ai agents require the integration of large language models and formal methods. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 82441–82459. PMLR, 2025.
- [113] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv:2307.13854*, 2023.
- [114] Yanfang Zhou, Yuntao Liu, Xiaodong Li, Yongqiang Zhao, Xintong Wang, Jinlong Tian, Zhenyu Li, and Xinhai Xu. Metagent-p: A neuro-symbolic planning agent with metacognition for open worlds. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.